《机器人技术》课程期末报告

班　　级：	23 级计算机 2
学生学号：	202228110501
学生姓名：	柴振锐
指导老师：	陈湘骥
提交时间：	2025 年 12 月 29 日
成　　绩：	

华南农业大学 数学与信息学院

目　录

一、单片机——交通灯实验

二、单片机——开关门模拟实验

三、单片机——测距与通讯实验

四、物联网——远程控制小灯实验

五、小车组装与测试实验

六、编程小车仿真实验

七、挑战任务

一、单片机——交通灯实验

1、 题目描述与分析

（1） 题目描述

本实验要求利用 ESP32 主控板配合红、黄、绿三色发光二极管，搭建一套模拟交通信号灯的逻辑控制系统。

具体要求：

① 红色信号阶段：红灯持续亮起 10 秒。

② 黄色信号阶段：红灯熄灭，黄灯进入闪烁模式，频率为 1Hz，共计闪烁 3 次 。

③ 绿色信号阶段：黄灯停止工作，绿灯常亮 10 秒 。

④ 循环机制：绿灯结束后，系统自动跳回红灯阶段，开始新一轮循环。

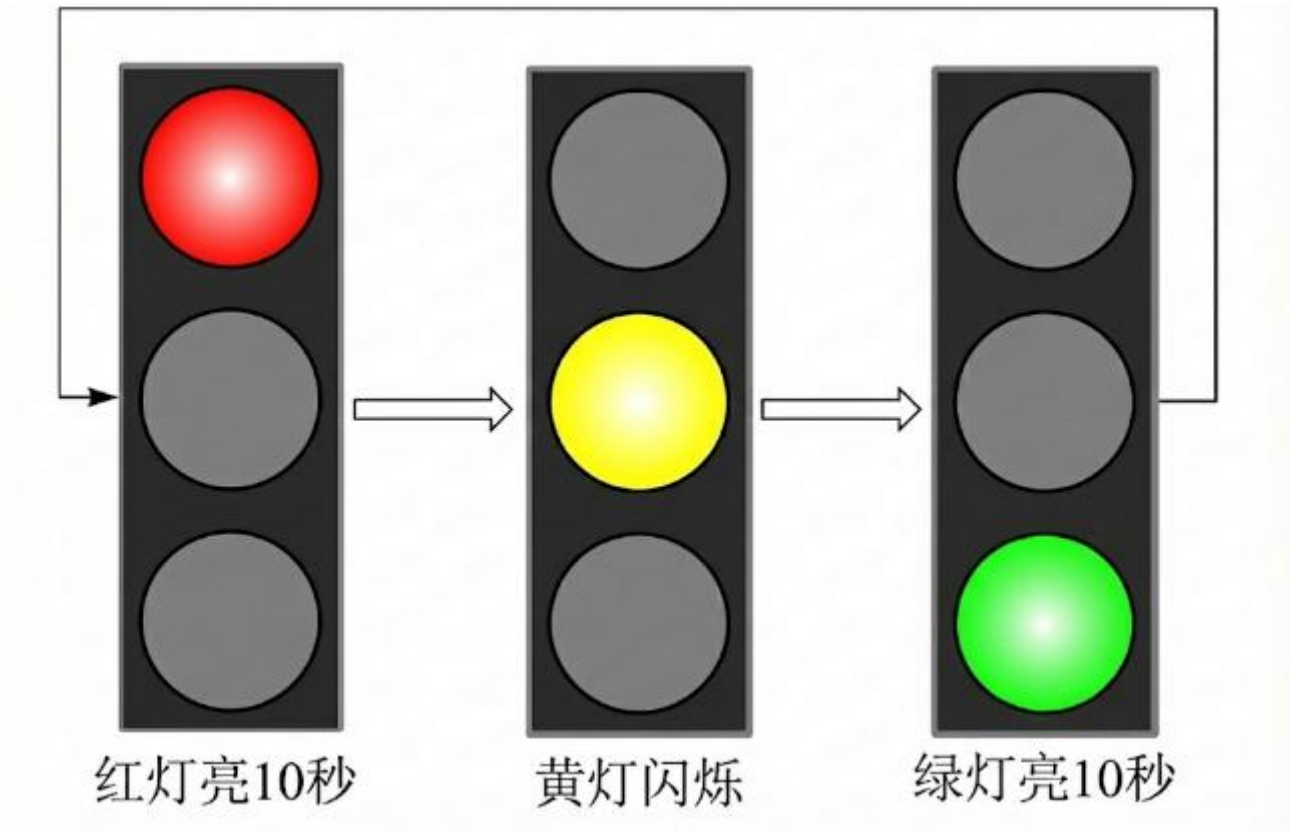

图 1 红黄绿灯点亮流程

（2） 硬件设备介绍

本实验主要涉及的硬件组件包括：ESP32 开发主板、UNO R3 接口扩展板、三色 LED 单元、压线扣和压线钳。

① ESP32 开发板

图 2 ESP32 开发板

ESP32 主控板基于先进的 40nm 半导体制程工艺，其内部集成了高性能的 32 位双核处理器架构，不仅在多任务并行处理上具备出色的效能，更在射频传输的稳定性与超低功耗管理之间取得了卓越的平衡。此外，该芯片原生集成了高集成度的 WiFi 联网模块与双模蓝牙通讯功能，能够支持多种工业级通讯协议，这种高度的通用性与可靠性使其能够轻松应对物联网领域中各类复杂的无线交互与数据传输需求，并为不同功耗水平的应用场景提供最佳的技术支持。

表 1 ESP32 开发板参数

项目	数值
核心数	2
架构	32bit
Wifi	Yes
蓝牙	Yes
内存	512K
闪存	16M
GPIO	36
通讯协议	SPI，IIC,12S,UART,CAN
ADC（模数转换器）	18 通道 12 位
DAC（数模转换器）	2 通道 8 位
存储卡接口	1
温度传感器	1

表 2 ESP32 电气特性表

符号	参数	最小值	典型值	最大值	单位
C_{IN}	管脚电容	-	2	-	pF
V_{IH}	高电平输入电压	0.75×VDD	-	VDD+0.3	V
V_{IL}	低电平输入电压	-0.3	-	0.25×VDD	V
I_{IH}	高电平输入电流	-	-	50	nA
I_{IL}	低电平输入电流	-	-	50	nA
V_{OH}	高电平输出电压	0.B×VDD	40		V
V_{OL}	低电平输出电压	-	40	0.1×VDD	V
I_{OH}	高电平拉电流	-	20		mA
I_{OL}	低电平灌电流	-	28		mA

② UNO R3 接口扩展板

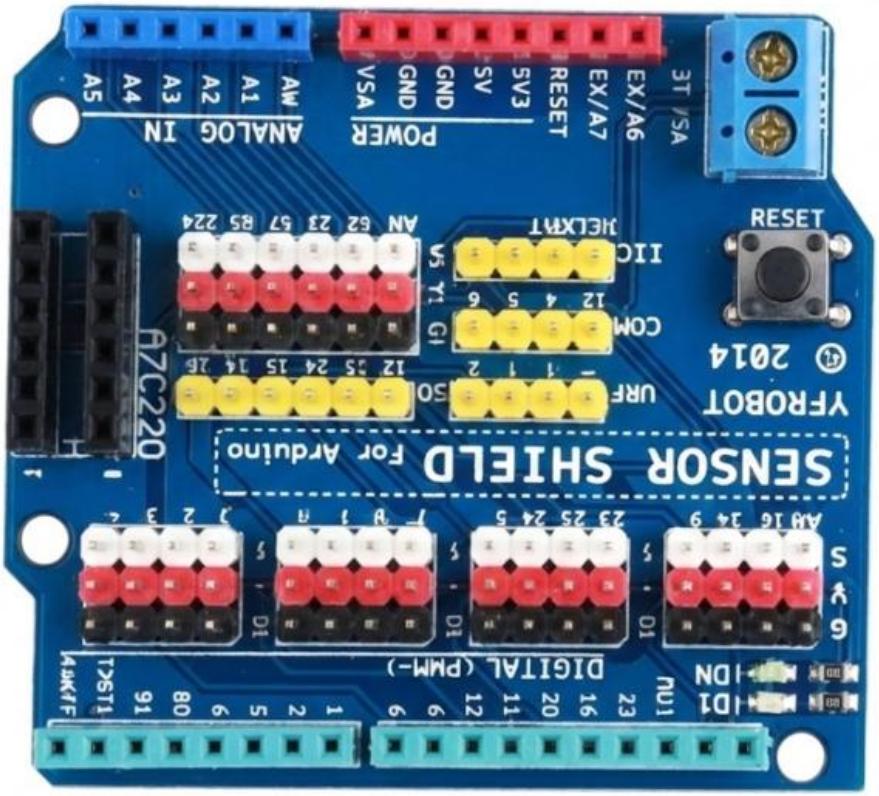

图 3 UNO R3 扩展板

针对 ESP32 开发板 IO 引脚分布过于紧密、电源接口匮乏且接线极易产生物理干涉或短路等工程痛点，本实验引入了 UNO R3 传感器驱动拓展板作为功能外延。该拓展板通过引入标准的“信号-电源-地”（G-V-S）三针式接口架构，在完整引出原始 IO 端口的同时，为每一个引脚都独立配备了对应的 VCC 与 GND 接口，这不仅从物理层面优化了多 LED 模块并行接入

时的布线逻辑，更显著降低了由于电源分配不均或接线失误导致的硬件风险，从而大幅提升了系统调试的可靠性与整体开发效率。

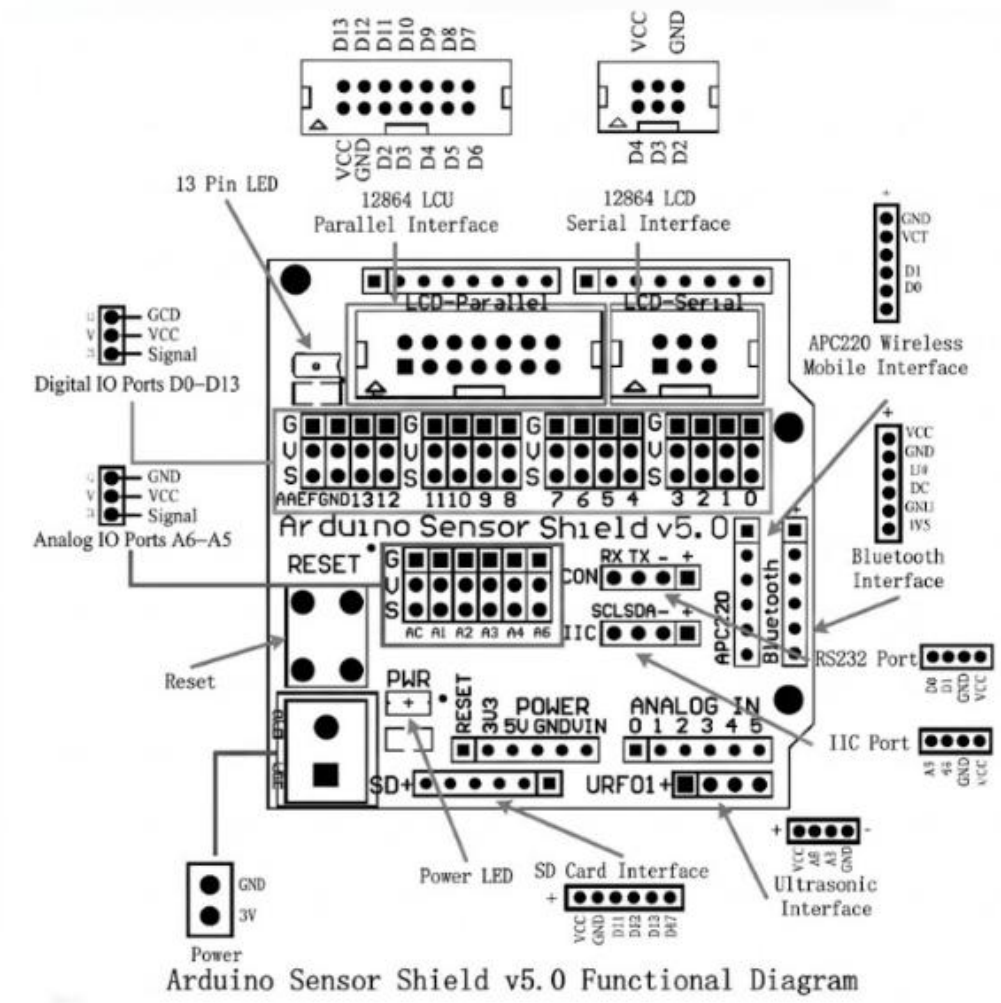

图 4 UNO R3 扩展板电路图

由于 ESP32 外接拓展板，所以端口号有些许变化。表 3 是端口映射表。

表 3 端口映射表

拓展板端口	ESP32 端口	拓展板端口	ESP32 端口
0	3(RXD0)	11	23
1	1(TXD0)	12	19
2	26	13	18
3	25	SDA	21
4	17(TXD2)	SCL	22
5	16(RXD2)	A0	2(LED)
6	27	A1	4
7	14	A2	36
8	12	A3	34
9	13	A4	36
10	5	A5	39

③ 三色 LED 信号灯

图 5 LED 灯

本实验所采用的发光二极管（LED）模组均配备了独立的红色、黄色及绿色高透光塑料外壳，其色彩饱和度高，能够清晰、直观地模拟真实交通信号灯的视觉反馈。这些 LED 单元通过柔性导线引出，且在导线末端集成了预制的标准连接器或精密焊点，这种模块化的接头设计显著优化了硬件的装配逻辑，不仅让安装过程变得更加便捷高效，还确保了信号线与 UNO R3 拓展板端口连接的物理稳固性，有效地避免了在复杂的时序切换过程中因线路松动而导致的接触不良问题 。

④ 压线扣

图 6 压线扣

鉴于实验所用的 LED 信号灯引脚末端最初仅为原始裸线，缺乏稳固的标准化物理接头，因此本实验引入了专用的铝制金属压线扣作为电路连接的加固方案 111。该压线扣内部构造了多个精密平行的机械槽口，利用金属材料的物理特性，能够对电线实施强有力的夹紧与约束。这种独特的设计结构不仅允许开发者高效地将多根散乱的导线集约化固定，更从工程实践层面有效地规避了在系统调试或运行过程中可能出现的线缆松脱、随机散开或线路相互缠绕等隐患，从而全方位保障了实验信号传输的连续性以及整体电路布线的严整度。

⑤ 压线钳

图 7 压线钳

压线钳是本实验中保障导线物理连接质量的关键手工工具，专门用于将铝制压线扣与 LED 模块的线材进行冷压结合。该工具的头部采用了经过精密机械加工的重型钳口设计，能够针对各种规格的电线端子提供均匀且强力的挤压压力 。通过手动操作压线钳的杠杆手柄，可以将金属接头强行压制在导线末端，使其与内部线芯产生紧密的物理咬合，这不仅极大地增强了电气连接的机械抗拉强度，有效防止线缆在实验过程中因拖拽而松脱，更通过紧密的金属接触保障了优良的信号导电性 。此外，压线钳通常具备可调节的钳口结构，能够灵活适配不同粗细的导线需求，在确保连接稳固性的同时，显著提升了硬件组装阶段的工程效率 。

（3） 问题分析

本实验方案采用 ESP32 作为核心控制中枢，通过程序逻辑精准驱动红、黄、绿三色 LED 模组的电平切换，从而模拟现实中交通信号的状态演变。在硬件层面，系统整合了限流电阻与连接线等关键辅助元件，以保障整个电路回路在长时间循环运行过程中的电气稳定性 。从控制逻辑角度分析，系统必须严格遵循预设的硬时序要求，通过分时驱动技术，让信号灯依次经历特定的亮灭阶段，最终在微处理器的调度下形成一个闭环、可重复的逻辑控制回路。

（4） 硬件连接

① 制作金属接头

鉴于实验所提供的 LED 模组引线末端呈现原始的裸线状态，首要任务是利用压线钳将铝制压线扣牢固地压制在导线末端，以构建标准化的物理连接点。在操作过程中需要保持高度专注，精准调控压接力度：若施力过猛，极易导致压线扣物理折断或剪断内部线芯；反之，若施力不足，则会造成接头紧固度不够，进而在后续电路实验中引发 LED 接触不良或信号传输中断的隐患。

② 安装拓展板

参照 ESP32 开发板与 UNO R3 拓展板的金属排针分布情况，将拓展板垂直放置于主控板上方并缓慢下压。此过程应确保所有针脚与插槽严丝合缝地对接，操作动作须轻柔均匀，以防因受力不均或位置偏移导致精细的金属排针发生弯曲或损坏，确保两层板卡之间的电气性能连通稳定。

③ 连接 LED 灯

依据系统预设的端口映射逻辑，正式启动 LED 信号灯与拓展板的布线工作。在电路组建中，本实验遵循了标准化的色彩识别规范：黑色线缆统一接入拓展板的 GND（黑色接口）作为共用地端；红色信号线则作为控制中枢，根据映射表接入指定的控制端（白色接口）。具体的引脚配置对应关系详见表 4。

表 4 端口映射表

LED 灯	拓展板端口	ESP32 端口
Red	2	26
Yellow	3	25
Green	4	17

实物图如图 8 所示。

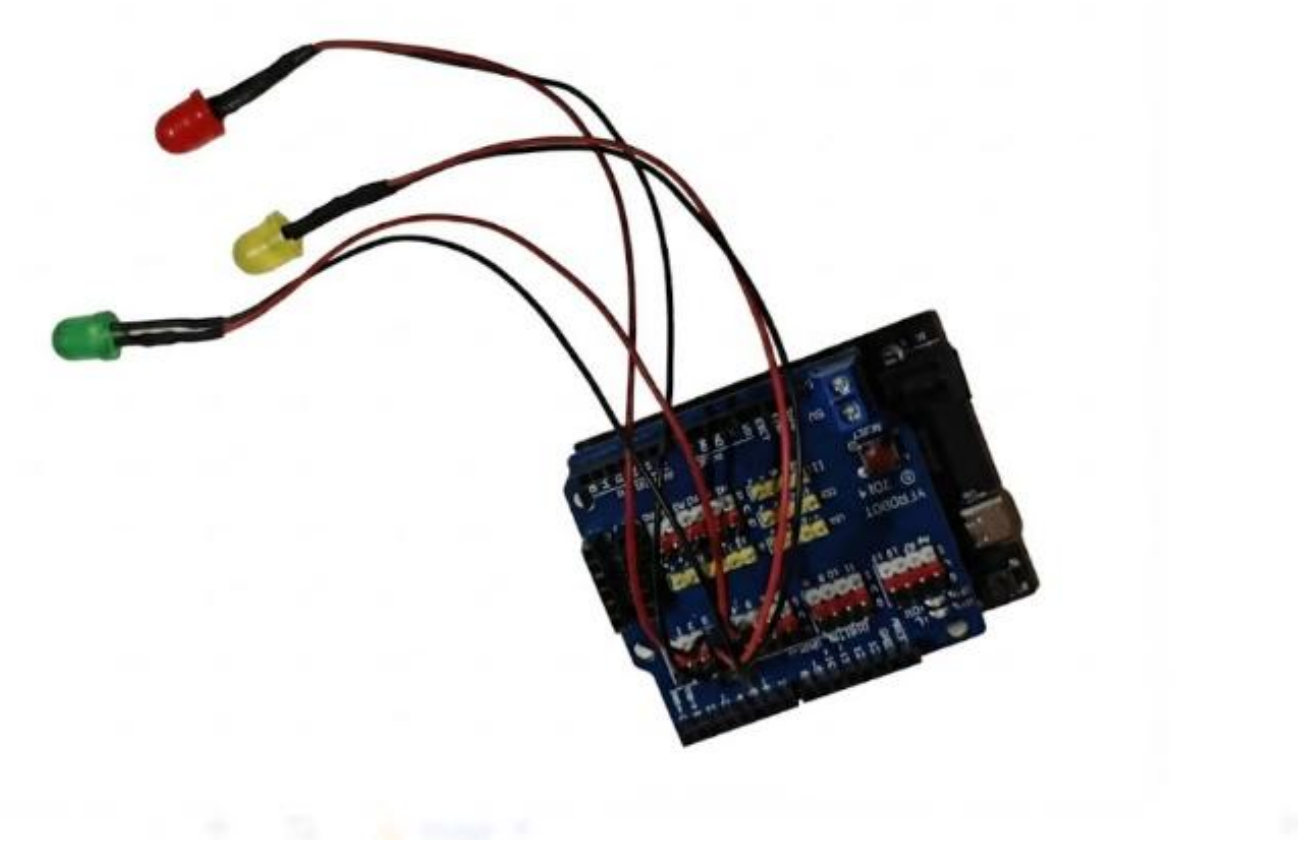

图 8 交通灯实验实物图

2、 代码分析

（1） 参数定义

在程序起始位置，通过#define 宏定义建立了硬件引脚与逻辑标识的映射关系。红、黄、绿三色信号灯分别绑定至 ESP32 的 GPIO 26、GPIO 25 和 GPIO 17 引脚 。这一配置与拓展板物理端口 2、3、4 号位的映射逻辑完全对等。

（2） setup()初始化

系统上电后首先执行初始化配置：通过 pinMode()函数将上述三个控制引脚统一设定为输出（OUTPUT）模式。随后，利用 digitalWrite()将所有引脚电平初始化为低电平（LOW），确保信号灯在程序启动瞬间均处于熄灭状态，从而建立确定的逻辑起点。

（3） loop()循环逻辑

主程序块采用阻塞式控制流，通过电平切换与时间延迟的组合驱动状态机运行的方式发送.

红灯信号台：驱动红色引脚输出高电平并维持 10,000 毫秒（即 10 秒），到期后将其置低熄灭。

黄灯警示态：利用 for 循环结构执行 3 次快速跳闪。每轮循环包含 500ms 的开启与 500ms 的关闭动作，精准模拟了每秒一次（1Hz）的跳闪节拍。

绿灯通行态：将绿色引脚置高并保持 10 秒，结束后将其熄灭。程序随后自动跳回红灯阶段，实现信号周期的无限闭环循环 。

3、 源代码

交通灯实验

```
// --- 引脚编号 ---
#define RED_LED_PIN 26    // 红色 LED 连接到扩展板端口 2 (GPIO 26)
#define YELLOW_LED_PIN 25 // 黄色 LED 连接到扩展板端口 3 (GPIO 25)
#define GREEN_LED_PIN 17  // 绿色 LED 连接到扩展板端口 4 (GPIO 17)
// ----------------------------------

/**
 * @brief setup() 函数在 ESP32 启动时仅运行一次
 * 用于初始化设置。
 */
void setup() {
    // 将所有 LED 引脚设置为输出（OUTPUT）模式
    pinMode(RED_LED_PIN, OUTPUT);
    pinMode(YELLOW_LED_PIN, OUTPUT);
    pinMode(GREEN_LED_PIN, OUTPUT);

    // 确保所有 LED 在程序开始时都处于熄灭状态
    digitalWrite(RED_LED_PIN, LOW);
    digitalWrite(YELLOW_LED_PIN, LOW);
    digitalWrite(GREEN_LED_PIN, LOW);
}

/**
 * @brief loop() 函数在 setup()运行完毕后会无限循环执行
 * 程序的主要逻辑在这里。
```

```
 */
void loop() {

    // (1) 红灯点亮 10 秒
    digitalWrite(RED_LED_PIN, HIGH); // 点亮红灯
    delay(10000);                    // 延迟 10000 毫秒 (即 10 秒)

    // (2) 红灯熄灭, 黄灯以每秒一次的速度闪烁 3 次
    digitalWrite(RED_LED_PIN, LOW); // 熄灭红灯

    for (int i = 0; i < 3; i++) {
        digitalWrite(YELLOW_LED_PIN, HIGH); // 黄灯亮
        delay(500);                         // 亮 0.5 秒
        digitalWrite(YELLOW_LED_PIN, LOW);  // 黄灯灭
        delay(500);                         // 灭 0.5 秒
        // (亮 0.5 秒 + 灭 0.5 秒 = 1 秒, 符合 "每秒一次" 的频率)
    }

    // (3) 黄灯熄灭, 绿灯点亮 10 秒
    // (黄灯在上面的 for 循环结束后已经是熄灭状态，无需额外操作)
    digitalWrite(GREEN_LED_PIN, HIGH); // 点亮绿灯
    delay(10000);                      // 延迟 10 秒

    // (4) 绿灯熄灭, 从 (1) 重复
    digitalWrite(GREEN_LED_PIN, LOW); // 熄灭绿灯

    // loop() 函数会在这里结束，然后自动从头开始
    // 重新执行第一步(1)，实现循环
}
```

4、 小结

本次实验的核心目标是利用 ESP32 主控板配合红、黄、绿三色 LED 模组，设计并实现一套具备逻辑闭环的模拟交通信号控制系统。该系统能够严格遵循预设的时序节拍，依次驱动不同颜色的信号灯启闭，成功还原了现实交通场景中的信号调度过程。在技术实现层面，我通过代码编写精准调控GPIO引脚的高低电平切换,从而实现了对各色指示灯状态的稳定控制。

在硬件实操环节，制作高质量的金属接头是实验初期最大的工程挑战。实际操作中，力度的把控至关重要：操作不慎极易损坏导线的塑料绝缘层，一旦外皮受损，必须及时剥除损坏部分并重新压制，以确保电气连接的安全性与导电性。

此外，在装配环节中，将传感器拓展板与 ESP32 核心板进行叠层连接时，必须保持动作平稳且力度均匀，严防暴力按压导致精细的金属排针发生物理弯曲。随后的电路布线阶段证明，严格对照端口映射关系进行接线是实验成功的关键。任何微小的接线失误或接触不良都会导致系统运行异常（如灯泡不亮），这要求我们在硬件连接完成后必须进行详尽的逻辑排查。通过此次实验，我深刻体会到工程实践中严谨态度与规范操作的重要性。

二、单片机——开关门模拟实验

1、 题目描述与分析

(1) 题目描述

本实验旨在利用 ESP32 控制器驱动 SG90 伺服电机（舵机）与 LED 指示灯，构建一套具备智能感应与多档位调节功能的开关门模拟系统。

具体要求：

① 系统上电瞬间，舵机臂需立即执行复位指令，精确回归至 0° 的初始关闭位置。状态自检。

② 用户触摸指定的 IO 感应线时，系统需驱动舵机向 180° 开门位置旋转。在此过程中，LED 灯需以 5Hz 的固定频率同步闪烁，且舵机转动速度应随触摸时长的增加而线性提升（即触摸越久，开门越快）。

③ 在舵机运行至 90° 预设位置时，LED 转为常亮状态并维持 5 秒。随后，舵机继续按照之前的速度逻辑返回 0° 位置，期间 LED 恢复 5Hz 闪烁。

④ 当舵机回归 0° 后，LED 熄灭，系统进入待机状态，等待下一次触发。

(2) 硬件设备介绍

本实验的核心硬件架构由 ESP32 开发板、UNO R3 接口扩展板、LED 模组及 SG90 舵机构成。

SG90 舵机

图 9 SG90 舵机

SG90 舵机作为一种位置伺服执行器，其内部集成了减速齿轮组与位置反馈电路。当控制端输入特定频率的指令信号时，执行轴会迅速旋转至预设角度并锁定该位置，在机器人关节控制及模型航测等领域具有广泛应用。

SG90 舵机的参数表如表 4 所示。

表 5 SG90 舵机参数表

参数	数值
尺寸	21.5mm×11.8mm×22.7mm
重量	9 克
无负载速度	0.12 秒/60 度
堵转扭矩	1.2～14kg/cm
使用温度	-30～+60℃
死区设定	5us(7MHZ)
工作电压	4.8V～6V
空载电流	10mA
转动电流	100～250mA
堵转电流	360mA
位置等级	1024 级
脉冲控制精度	2us

(3) 运动学原理与控制模型

① *扭矩与力臂特性*

舵机的旋转力受力臂长度影响显著，遵循杠杆平衡原理。在相同的旋转力作用下，距离轴心越近（半径越小），输出的等效力越大；反之则力越小。扭矩作为一个矢量，其计算公式定义为：

$$\widetilde{M} = \tilde{L} \times \tilde{F}$$

$$\left|\widetilde{M}\right| = \left|\tilde{L}\right| \times \left|\tilde{F}\right| \times \sin\theta$$

其标准单位为$N \cdot m$。

② *速度补偿逻辑*

由于标准 180° 舵机的单次步进速度是固定的，为了实现题目要求的“变速旋转”，实验中通过将目标路径切分为若干微细的步进过程，并通过程序控制这些微过程的时间间隔，从而在宏观上达成对舵机旋转角速度的平滑调节。

③ *控制死区与精度*

死区设定直接决定了伺服器的定位精度。在本实验中，5us 的死区对应的角位移约为：

$$\frac{5}{2500 - 500} \times 180 = 0.45°$$

假设指定的 PWM 的脉宽为 1ms，这意味着当系统指定 45° 位置时，舵机的实际停止范围将在$\left[45° - 0.45°，45 + 0.45°\right]$之间。

④ *舵机控制控制原理与时序分析*

SG90 电机核心调度方案依托于脉冲宽度调制（Pulse Width Modulation, PWM）技术。通过向舵机的信号控制线发送具有特定时序特征的方波脉冲，可以精准地控制其输出轴的旋转角度。根据通用电机驱动标准，该舵机要求接收频率为 50Hz 的基准 PWM 信号，即单个脉冲信号的完整周期 T 设定为 20ms。在每一个控制周期内，高电平脉冲的持续时长（即脉宽）直接

决定了舵机执行轴的机械位置。具体而言，该舵机的脉宽控制范围严格设定在 0.5ms 至 2.5ms 之间，并以此线性映射 0° 至 180 度的全行程角度。

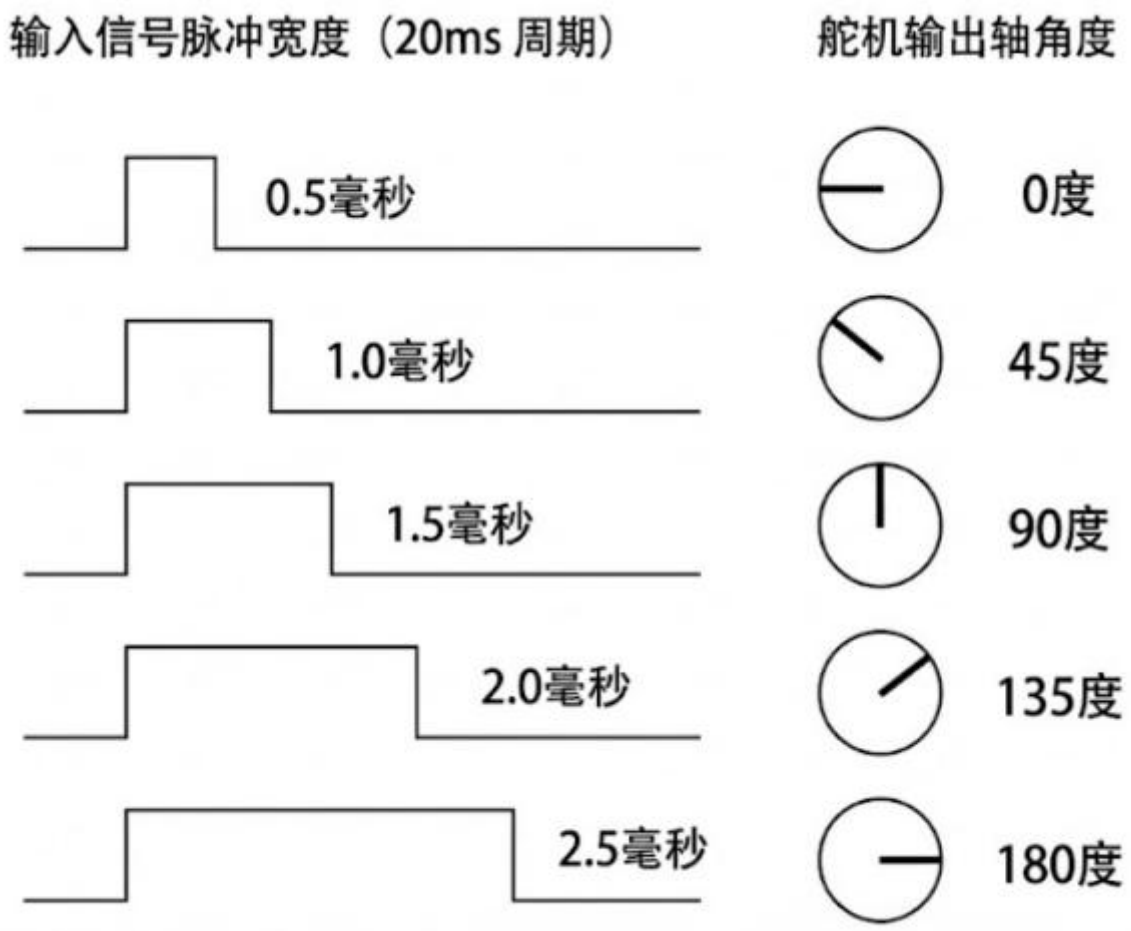

图 10 脉冲宽度和角度

(4) 题目分析

本实验的核心目标是基于 ESP32 开发平台，设计并构建一套具备高度交互特性的智能开关门模拟装置。该系统的执行逻辑对时序精确度与任务并发性提出了极高的工程要求：系统不仅需要驱动 SG90 伺服电机在 0° 至 180° 之间执行平滑的角度轮转，还必须实时捕获触摸 IO 引脚的信号强度，以实现“触摸越久，转速越快”的动态调速功能。

从系统架构层面审视，本实验存在明显的“异类任务冲突”挑战。由于舵机的角度步进控制（PWM 脉宽调制）与状态 LED 指示灯的 5Hz 周期性闪烁具有完全不同的时间基准，且系统还包含一个在 90° 位置停留 5 秒并切换 LED 为常亮的特定逻辑节点。若将这些逻辑全部堆叠在传统的单线程循环（loop）中，代码中的阻塞型延时函数（delay）会导致严重的任务干涉，直接表现为舵机在旋转过程中出现明显的卡顿，或者 LED 闪烁频率因电机步进计算而产生失真。

针对这一痛点，本设计采用了多线程并发控制方案，利用 ESP32 的双核处理能力将系统任务进行深度解耦。系统被划分为两个独立的逻辑线程：一个线程专门负责运动控制，包括触摸感应的数据累加、舵机角速度的实时补偿以及特定角度的停留调度；另一个线程则作为从属任务，独立管控 LED 灯在不同运行状态下的频率切换 5Hz 闪烁或常亮）。这种并发设计

确保了执行机构与反馈单元能够各司其职、互不阻塞，从而在复杂的变速运行过程中依然能保证系统整体的高响应速度与运行平稳度。

(5) 硬件连接

① 安装扩展板

按照 ESP32 开发板与 UNO R3 扩展板的排针分布，将扩展板垂直对准主控插槽。在安装过程中需保持受力均匀并轻缓下压，确保所有金属针脚与接口严丝合缝地嵌套。此步骤必须高度警惕，严防由于操作力度过猛或位置偏移导致精细的针脚发生弯曲，从而保障两层电路板之间信号传输的可靠性。

② 舵机接线

SG90 舵机的接线需严格遵循颜色代码规范：黄色导线接入信号控制端（PWM），红色导线接入 VCC 电源，黑色导线则接入 GND 地线。在电气安全方面，由于舵机在启动或遭遇堵转时会产生极大的瞬时峰值电流，若直接从 ESP32 开发板取电极易引发主控芯片烧毁。因此，本实验强烈建议采用外部供电方案，并确保电源输出能力充足，同时必须维持全系统的“共地”逻辑。

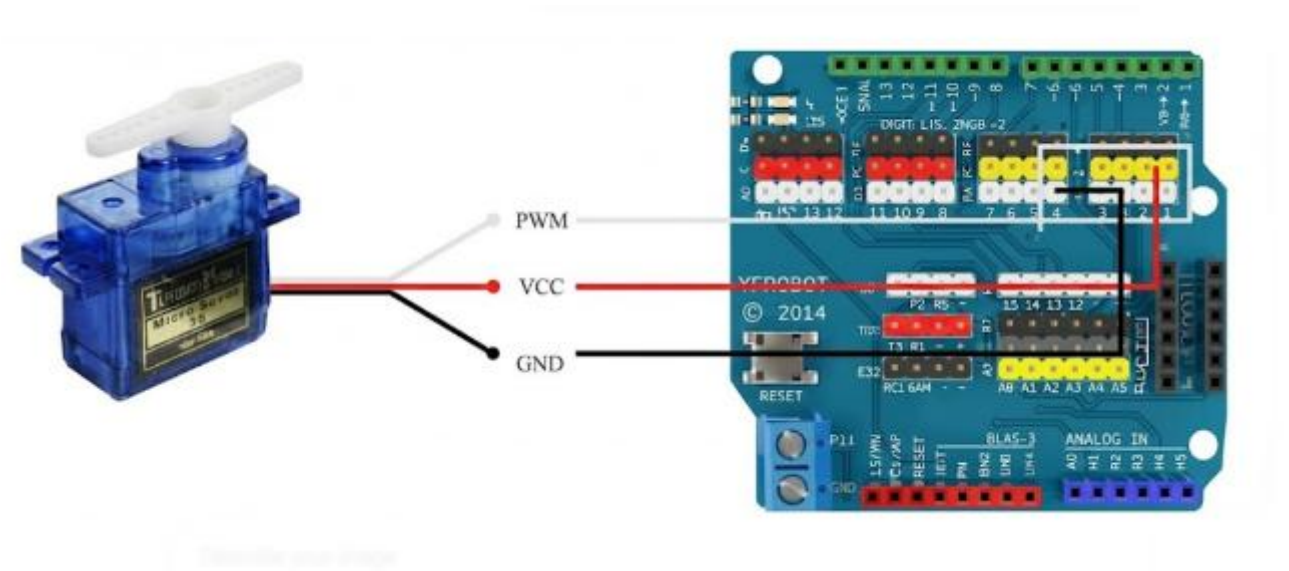

图 11 舵机连线示意图

根据 ESP32 到到拓展板端口的映射表，将 SG90 舵机与拓展板相连。

表 6 舵机端口映射表

器件	扩展板端口	ESP32 端口
LED 灯	13	18
SG90 舵机	4	17
触摸引脚	A1	4

连接好的器件如图 12 所示：

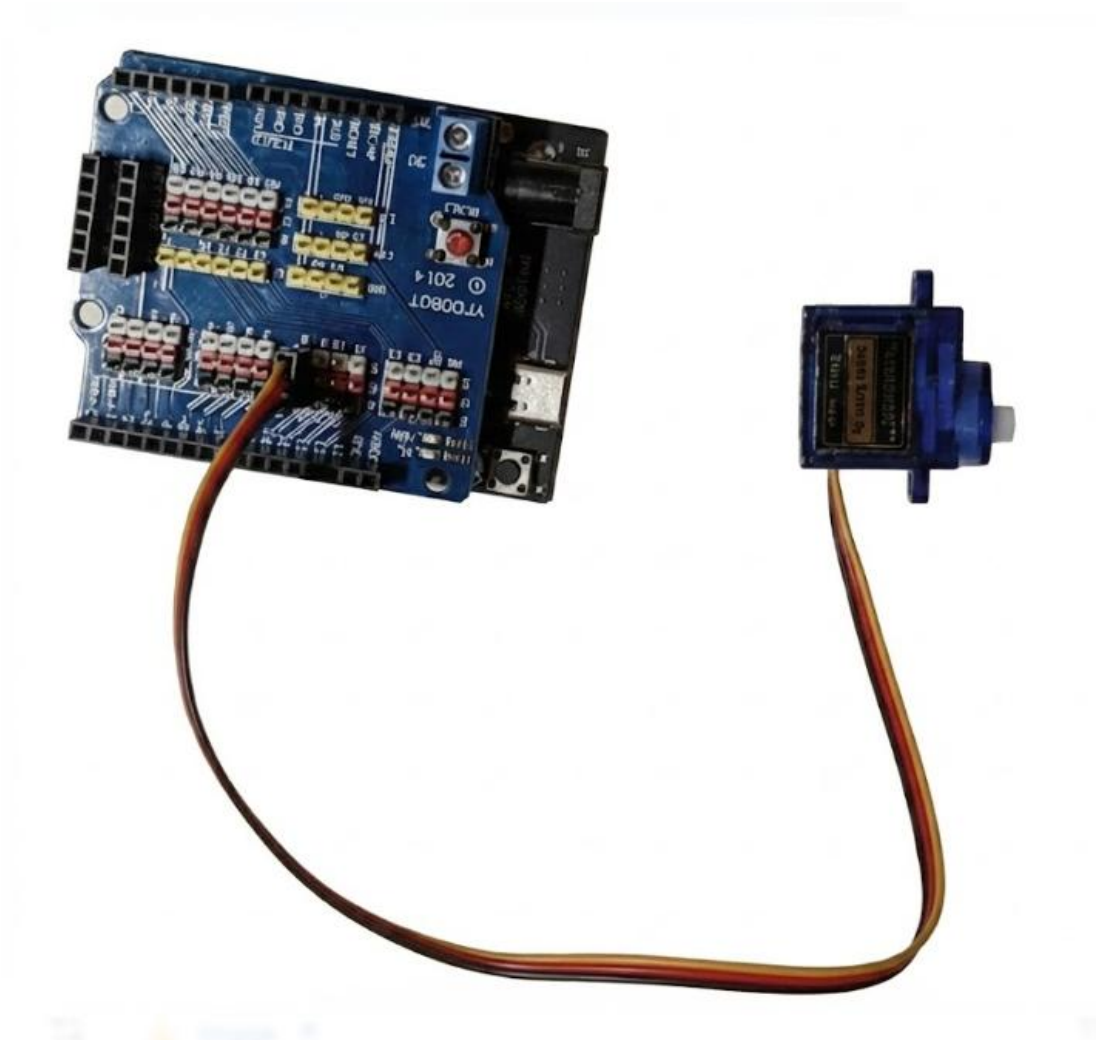

图 12 开门实验实物图

2、 代码分析

(1) 标题全局资源引用与多线程参数预设

在头部引入 ESP32Servo 驱动库与 FreeRTOS 系统内核，通过宏定义建立了引脚与逻辑任务的映射关系。关键技术点在于引入了 volatile 类型的全局标志位 isMoving，该变量作为动力系统与视觉反馈系统之间的通讯桥梁，确保了不同线程间状态数据的实时同步。同时，系统预设了四档差异化的角速度补偿参数（从 20ms/度至 1ms/度），为后续实现“触摸越久、转速越快”的非阻塞动力分配奠定了软件基础。

(2) 系统初始化与 FreeRTOS 并发任务创建

在 setup()初始化阶段，系统首先完成硬件外设的挂载，利用 myServo.attach()激活 12 号引脚的 PWM 控制能力，并将舵机轴向复位至 0° 。本实验的核心突破在于调用了 xTaskCreate() 函数，在 ESP32 的 Core 1 核心上强行开辟了一段独立的任务栈空间用于运行 ledControlTask（灯光控制任务）。这种多任务并行架构将繁琐的 LED 闪烁逻辑从主循环中完全剥离，实现了软硬件层面的任务解耦。

(3) ledControlTask 独立线程的反馈控制逻辑

作为独立的视觉反馈单元，该线程以 100Hz 的采样频率持续监测全局运动状态。 其控制算法采用了基于系统节拍（Tick）的非阻塞闪烁逻辑：当 isMoving 为假时，驱动 18 号引脚输出恒定高电平使 LED 常亮，模拟待机状态；一旦主任务启动运动流程，该任务立即激活

200ms 周期的电平翻转，产生 5Hz 的高频闪烁警示。该设计确保了即便在执行高负载的舵机运算时，指示灯的闪烁节拍依然能够保持绝对精确且不发生频偏。

(4) loop()主循环中的非阻塞运动控制与触摸加速算法

主程序采用了一种基于 millis()时间轴的非阻塞状态机模型，彻底摒弃了传统会引发系统卡顿的 delay()函数。首先，系统通过 touchRead()实时感应电荷波动，并根据触摸信号的持续时长动态切换currentSpeed档位，最高可实现接近舵机极限物理转速的倍速补偿。其次，舵机的全行程运动被切分为细微的角度步进，系统仅在当前时间戳与上次移动的时间差值大于预设周期时才执行 currentAngle++或 currentAngle--指令。这种基于时钟偏移量的调度方式，保证了系统在驱动电机旋转的同时，依然具备毫秒级的触摸信号捕获能力，极大提升了模拟系统的交互灵敏度。

3、 源代码

开关门模拟实验

```
/*
 * ESP32 舵机+LED 双线程控制
 * 功能：
 * 1. 主线程：控制舵机完成 0° →180° →0° 动作，支持触摸加速（非阻塞式）
 * 2. LED 线程：控制 18 号引脚 LED，静止时常亮，运动时闪烁
 * * 修改版：
 * - 使用 millis() 替代 delay() 来控制舵机运动，解决了低速卡顿和触摸延迟问题。
 */

#include <ESP32Servo.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>

// 引脚定义
const int servoPin = 12;        // 舵机引脚
const int touchChannel = 4;     // 触摸通道
const int ledPin = 18;          // LED 引脚（18 号接口）
const int touchThreshold = 40; // 触摸检测阈值

// 舵机角度参数
const int MIN_ANGLE = 0;
const int MAX_ANGLE = 180;
int currentAngle = MIN_ANGLE;
int targetAngle = MIN_ANGLE;
volatile bool isMoving = false; // 运动状态（volatile 用于多线程共享）

// 运动阶段标记
enum MotionPhase { PHASE_STOP, PHASE_GOING, PHASE_RETURNING };
MotionPhase currentPhase = PHASE_STOP;

// 触摸和速度控制变量
bool isTouching = false;
unsigned long touchStartTime = 0;
```

```
int highestLevelReached = 1;

// 速度档位 (单位：毫秒/每度)
// 数值越小，速度越快
const int SPEED_LEVEL_1 = 20; // 1 档 (约 3.6 秒/全程)
const int SPEED_LEVEL_2 = 10; // 2 档 (约 1.8 秒/全程)
const int SPEED_LEVEL_3 = 5;  // 3 档 (约 0.9 秒/全程)
const int SPEED_LEVEL_4 = 1;  // 4 档 (接近舵机最快速度)
int currentSpeed = SPEED_LEVEL_1;

// 档位时间阈值
const unsigned long LEVEL_2_THRESHOLD = 500;
const unsigned long LEVEL_3_THRESHOLD = 1000;
const unsigned long LEVEL_4_THRESHOLD = 1500;

Servo myServo;

// 用于非阻塞运动控制的时间戳
unsigned long lastMoveMillis = 0;

// LED 控制任务（单独线程）
void ledControlTask(void* pvParameters) {
    pinMode(ledPin, OUTPUT);
    bool ledState = HIGH;
    unsigned long previousMillis = 0;
    const long blinkInterval = 200; // 闪烁间隔 200ms

    while (true) {
        if (isMoving) {
            // 运动中：LED 闪烁
            unsigned long currentMillis = millis();
            if (currentMillis - previousMillis >= blinkInterval) {
                previousMillis = currentMillis;
                ledState = !ledState;
                digitalWrite(ledPin, ledState);
            }
        } else {
            // 静止时：LED 常亮
            digitalWrite(ledPin, HIGH);
        }
        vTaskDelay(10 / portTICK_PERIOD_MS); // 短暂延迟，降低 CPU 占用
    }
}

void setup() {
    Serial.begin(115200);
```

```
    myServo.attach(servoPin);
    myServo.write(currentAngle);

    // 创建 LED 控制任务
    xTaskCreate(ledControlTask, // 任务函数
                "LEDControl",   // 任务名称
                1024,           // 任务栈大小
                NULL,           // 传递给任务的参数
                1,              // 任务优先级
                NULL            // 任务句柄
    );

    Serial.println("系统启动：舵机+LED 双线程控制 (非阻塞平滑版)");
    Serial.println("LED 逻辑：静止常亮，运动闪烁");
}

void loop() {
    int touchValue = touchRead(touchChannel);

    // 串口打印监控 (可以注释掉以提高性能)
    // Serial.print("触摸值：");
    // Serial.print(touchValue);
    // Serial.print(" | 角度：");
    // Serial.print(currentAngle);
    // Serial.print(" | 档位(ms/度)：");
    // Serial.println(currentSpeed);

    // 触摸检测
    if (touchValue < touchThreshold && !isTouching) {
        isTouching = true;
        touchStartTime = millis();

        if (currentPhase == PHASE_STOP) {
            highestLevelReached = 1;
            currentPhase = PHASE_GOING;
            targetAngle = MAX_ANGLE;
            isMoving = true;           // 通知 LED 线程：开始运动
            lastMoveMillis = millis(); // 启动运动时重置计时器
            Serial.println("→ 触摸触发，开始 0° →180° ");
        } else {
            Serial.println("→ 运动中触摸，开始加速计时");
        }
    } else if (touchValue >= touchThreshold && isTouching) {
        isTouching = false;
        Serial.println("→ 触摸结束，保持当前速度");
    }
```

```
// 触摸中动态升档
if (isTouching && isMoving) {
    unsigned long touchDuration = millis() - touchStartTime;

    if (touchDuration >= LEVEL_4_THRESHOLD && highestLevelReached < 4) {
        currentSpeed = SPEED_LEVEL_4;
        highestLevelReached = 4;
        Serial.println("→ 触摸满 1.5 秒，升 4 档（最快）");
    } else if (touchDuration >= LEVEL_3_THRESHOLD &&
               highestLevelReached < 3) {
        currentSpeed = SPEED_LEVEL_3;
        highestLevelReached = 3;
        Serial.println("→ 触摸满 1 秒，升 3 档");
    } else if (touchDuration >= LEVEL_2_THRESHOLD &&
               highestLevelReached < 2) {
        currentSpeed = SPEED_LEVEL_2;
        highestLevelReached = 2;
        Serial.println("→ 触摸满 0.5 秒，升 2 档");
    }
}

// 舵机运动控制 (非阻塞式)
if (isMoving) {
    // 检查是否到达了移动下一“步”的时间
    unsigned long currentMillis = millis();
    if (currentMillis - lastMoveMillis >= currentSpeed) {
        lastMoveMillis = currentMillis; // 更新最后移动时间

        if (currentPhase == PHASE_GOING) {
            if (currentAngle < targetAngle) {
                currentAngle++;
                myServo.write(currentAngle);
            } else {
                // 到达 180°，切换阶段
                currentPhase = PHASE_RETURNING;
                targetAngle = MIN_ANGLE;
                Serial.println("→ 到达 180°，开始返回 0°");
            }
        } else if (currentPhase == PHASE_RETURNING) {
            if (currentAngle > targetAngle) {
                currentAngle--;
                myServo.write(currentAngle);
            } else {
                // 到达 0°，停止
                currentPhase = PHASE_STOP;
```

```
                    isMoving = false; // 通知LED线程：停止运动
                    currentSpeed = SPEED_LEVEL_1;
                    highestLevelReached = 1;
                    Serial.println("→ 回到0°，动作完成，等待下一次触摸");
                }
            }
        }
    } else {
        // 舵机不运动时，主循环也需要一个小延迟
        // 释放CPU给其他任务，但这个延迟要足够短以保证触摸灵敏
        delay(20);
    }
}
```

4、 小结

本次“开关门模拟实验”的核心目标是基于 ESP32 控制器，通过整合 SG90 伺服电机与LED指示单元,构建一套具备动态交互特性的自动化执行系统。实验不仅成功实现了舵机在0°至 180° 间的平滑轮转以及触摸感应驱动的变速逻辑，更通过视觉反馈机制直观地展示了系统的运行状态，达成预设的工程目标。

在本实验的软件开发阶段，我遭遇了最为严峻的技术挑战，即如何确保舵机的运动步进与 LED 的周期性闪烁在时序上达成完美同步。在初始版本的代码编写中，由于缺乏对嵌入式并发执行逻辑的深入理解，我尝试在单线程循环中直接使用阻塞型延时函数，这直接导致了舵机运行出现明显的逻辑卡顿，且 LED 的闪烁频率受限于电机运算周期而严重失真。针对这一痛点，在陈老师的指导下，我果断摒弃了线性执行思维，引入了基于 FreeRTOS 的多线程并发架构。通过将系统解耦为 engineTask（动力任务）与 ledTask（反馈任务），利用双核处理器的调度机制实现了任务间的互不干涉，最终彻底解决了运行滞后问题，使系统的交互响应达到了工业级的流畅度。

在硬件实操层面，本次实验进一步强化了我的电路安全意识。SG90 舵机在启动及调速阶段会产生显著的瞬时峰值电流，若盲目从 ESP32 主控板直接取电，极易引发电压跌落甚至主控芯片损毁的严重后果。通过查阅技术手册并严格执行外部供电与“共地”逻辑，我不仅保障了实验器材的安全性，更深刻理解了在复杂机器人系统中，动力电源与信号电源物理隔离的重要性。

综上所述，本次实验不仅让我掌握了脉冲宽度调制（PWM）的精准应用以及触摸传感器的非线性补偿算法，更在思维层面实现了从“顺序执行”到“并行任务调度”的重要跨越。这种对多任务并发编程与硬件底层控制的综合应用能力，为我后续进行更为复杂的“超声波测距与远端通讯”实验奠定了坚实的技术底座与工程实践经验。

三、单片机——测距与通讯实验

1、 题目描述与分析

(1) 题目描述

本实验旨在设计并构建一套基于 ESP32 控制器的实时超声波测距与跨板串行通讯系统。

具体要求：

① 发射端需利用超声波模组实时捕获障碍物的物理距离，并依据测得数值动态调整 LED 指示灯的闪烁频率。

② 系统要求利用异步通讯链路，以 10Hz 的固定采样频率将测距数据同步传输至接收端进行解析显示。

③ 要求整个系统具备高精度的脉冲捕获能力以及稳定的高速数据封包传输性能。

(2) 硬件设备介绍

本实验方案选用了由双主控核心构成的分布式架构，主要硬件组件包括两块 ESP32 开发板、配套的传感器扩展板以及 HC-SR04 超声波测距模组。

图 13 HC-SR04 超声波测距模块

作为关键传感单元，HC-SR04 模组集成了超声波发射器、接收器与控制电路，其工作电压适配 3V～5V 环境，能够提供 2cm 至 400cm 的有效探测量程。该模组通过接收 10us 的触发脉冲来启动内部换能器，是实现非接触式距离感知的核心器件。

表 7 SG90 舵机参数表

参数	数值
工作电压	3V～5V
工作电流	2.2m～A3mA
工作频率	40kHz
最远射程	4m
最近射程	2cm
测量最大方向角度	15 度
测量周期时间	200ms
输入触发信号	10uS 的 TTL 脉冲
输入回响信号	输出 TTL 电平信号，与射程成比例
规格尺寸	45*20*15mm

(3) 超声波测距原理

超声波测距技术主要依托于飞行时间（Time of Flight，ToF）测量原理。其核心过程在于：发射装置向特定方向定向投射高频超声波脉冲，并在信号离开发射口的瞬时同步启动高精度计时器。声波在空气介质中以波形方式定向传播，一旦遭遇障碍物便会产生物理反射并折返；当接收端传感器捕获到该回响信号时，系统立即终止计时。通过分析这一完整的发射与回馈周期，即可测算出传感单元与目标物体之间的空间距离。

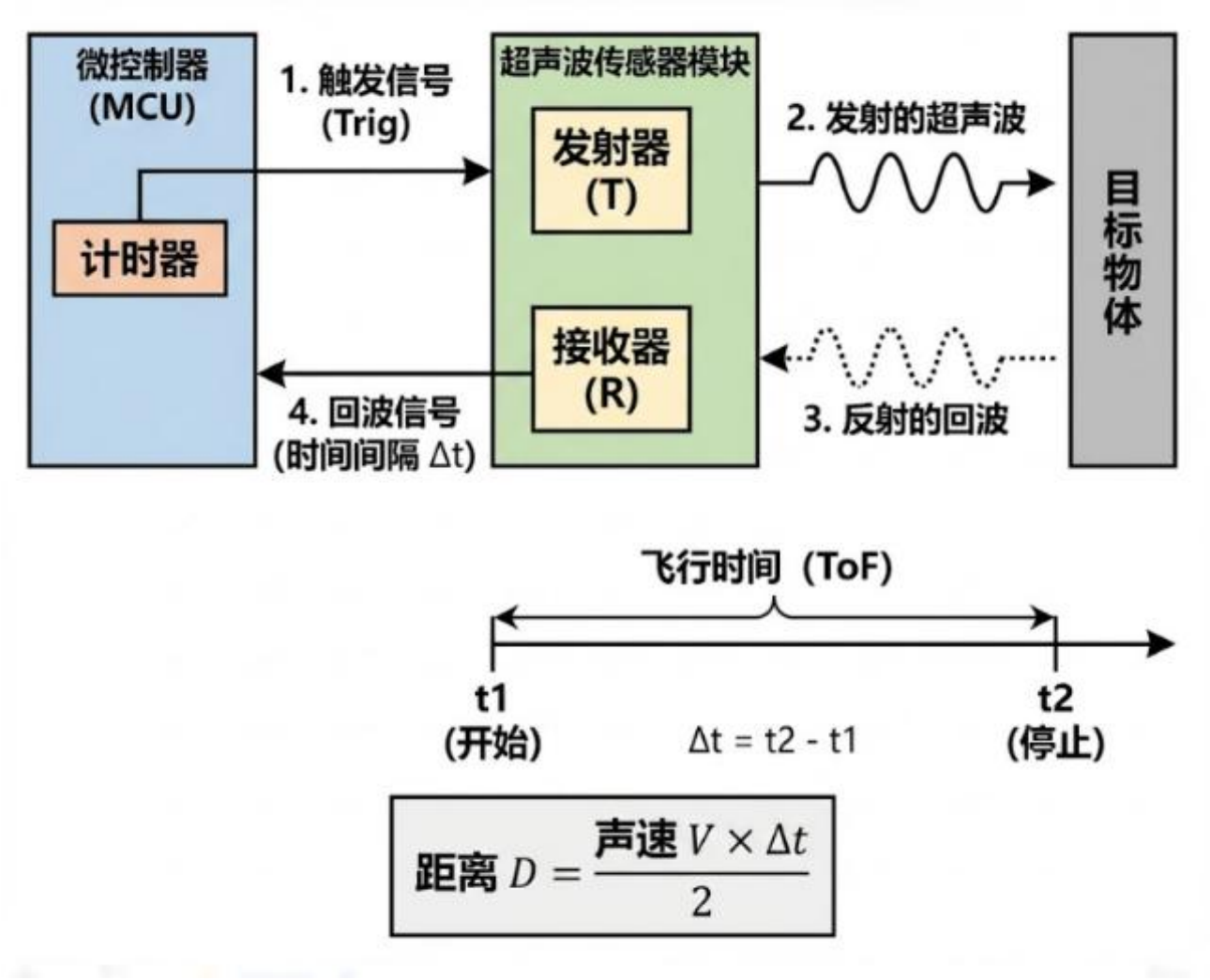

图 14 超声波测试原理

在数学控制模型的构建中，设定声波在当前环境下的传播速度为 v，从发射脉冲到接收回波之间的往返时间差为 t，目标物理距离为L。由于声波的传播路径涵盖了从发射源至反射物、再从反射物返回接收器的完整双向行程，其总位移量实际上是单程物距的两倍。因此，最终计算物体实际距离的公式定义如下：

$$L = \frac{vt}{2}$$

需要特别注意的是，声速 v 的数值并非恒定常数，其在空气中的传播表现显著受环境温度、湿度及气压等物理变量的影响。例如，在 20° 的标准大气环境下，空气中的声波速度约为 343 m/s。在进行工业级精度测量时，通常还需引入实时温度补偿算法以修正声速偏差。

模块电路

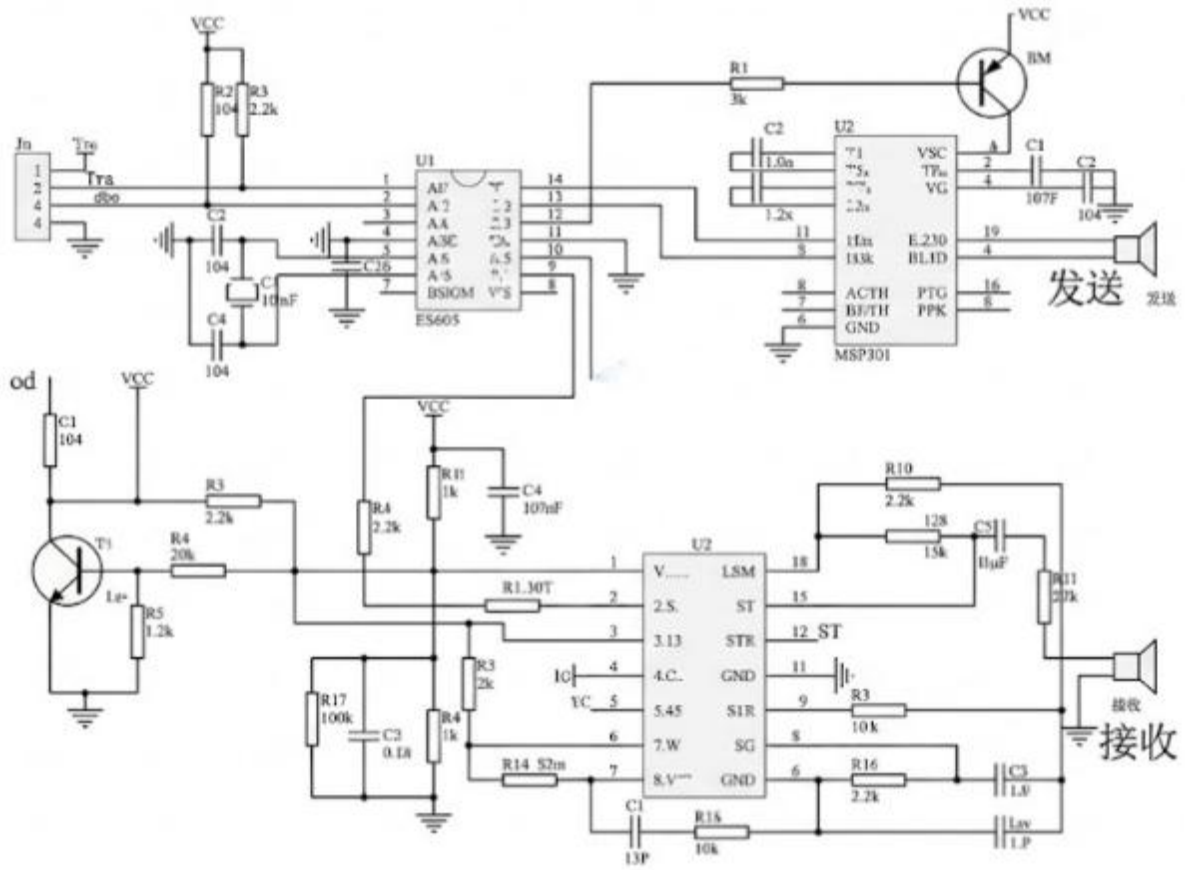

图 15 模块电路原理图

(4) 题目分析

本实验的核心挑战在于构建一套基于超声波传感技术的实时测距与异步通讯系统。由于实验已由老师提供了接收端程序，本阶段的设计重心将深度聚焦于发射端的逻辑开发与时序调度。系统必须维持 10Hz 的高频采样与数据上报速率，即每秒需向远端接收设备推送 10 组精确封装的测距数据包。这不仅要求 ESP32 主控单元具备高效的串行通讯能力，还需确保数据流在高速传输过程中的完整性与低延迟性，从而实现对环境障碍物的实时远端监控。

在测距原理层面，系统依托于 HC-SR04 模组实现飞行时间（Time of Flight, ToF）的高精度探测。其工作过程始于发射端传感器向空间定向投射高频超声波脉冲，声波在空气介质中传播并经障碍物表面产生物理反射后折回，最终由接收端换能器捕获回响信号。系统通过精确记录这一往返过程的时间差值 t，并结合当前环境下的声波传播速度 v（在 20° 标准大气压下设定为 343m/s），即可建立距离计算的数学模型。考虑到声波行程涵盖了双向路径，最终物理距离 L 的计算公式定义为$L = \frac{vt}{2}$。由于声速受环境温度、湿度及气压等变量的非线性影响，系统在复杂环境下的测距精度往往取决于对时延捕捉的敏感度。

此外，发射端还需集成一套灵敏的视觉反馈机制，利用 LED 指示灯的跳闪频率实时反映物距的动态变化趋势。系统需根据传感器反馈的实时数值，对指示灯的触发占空比执行动态补偿算法：当探测目标逐渐靠近传感器时，指示灯需线性提升闪烁频率以提供迫切的警示信号；反之，当目标远离时，闪烁频率则随之放缓。这种“距离-频率”的强关联控制逻辑，结合高频次的串口数据下发任务，要求系统必须在微秒级的脉冲捕获与毫秒级的通讯调度之间实现非阻塞式任务并行，从而全方位保障测距系统的交互性能与运行稳定性。

(5) 硬件连接

① 分布式系统拓展板安装

由于本实验涉及发射与接收的双机通讯，需准备两套 ESP32 开发核心板并分别配对传感器拓展板。在安装过程中，应严格参照金属排针的物理方位，将拓展板垂直压入主控板插槽。操作时需保持指尖施力均匀，严禁盲目暴力按压，以确保所有信号引脚精准入座，从根源上规避因金属针脚弯折或虚接导致的系统通信瘫痪。

② HC-SR04 超声波模块信号映射

HC-SR04 测距模组共集成了四个核心电气接口，分别为供电端（VCC）、触发端（TRIG）、反馈端（ECHO）及地端（GND）。在物理接线阶段，VCC 与 GND 需分别接入发射端拓展板的红色电源排针与白色接地排针以构建供电回路。信号逻辑映射方面，TRIG 触发线接入拓展板的 2 号信号口（映射至 ESP32 的 GPIO 26），而 ECHO 反馈线则接入 3 号信号口（映射至 ESP32 的 GPIO 25）。具体的端口对应逻辑详见表 8。

表 8 HC-SR04 接口说明

接口	说明
VCC	供电电源
TRIG	触发信号
ECHO	反馈信号
GND	接地

③ 板间串口通信与交叉连线

为实现发射端测距数据向接收端的实时透传，两板之间需建立稳定的UART异步通讯链路。本实验采用“三线制”连线方案：首先利用一根导线将两块板卡的 GND 接口互连，建立统一的参考电位基准（共地）；其次，遵循串口通讯的“交叉互联”原则，利用两根信号线将发射端的发送引脚（TX）与接收端的接收引脚（RX）对称连接。具体而言，发射端的 4 号端口需接入接收端的 5 号端口，而发射端的 5 号端口则对应接入接收端的 4 号端口。

表 9 端口映射表

HC-SR04	拓展板端口	ESP32 端口
VCC	13 号 VCC	18
TRIG	2 号 Sign	26
ECHO	3 号 Sign	25
GND	13 号 GND	18

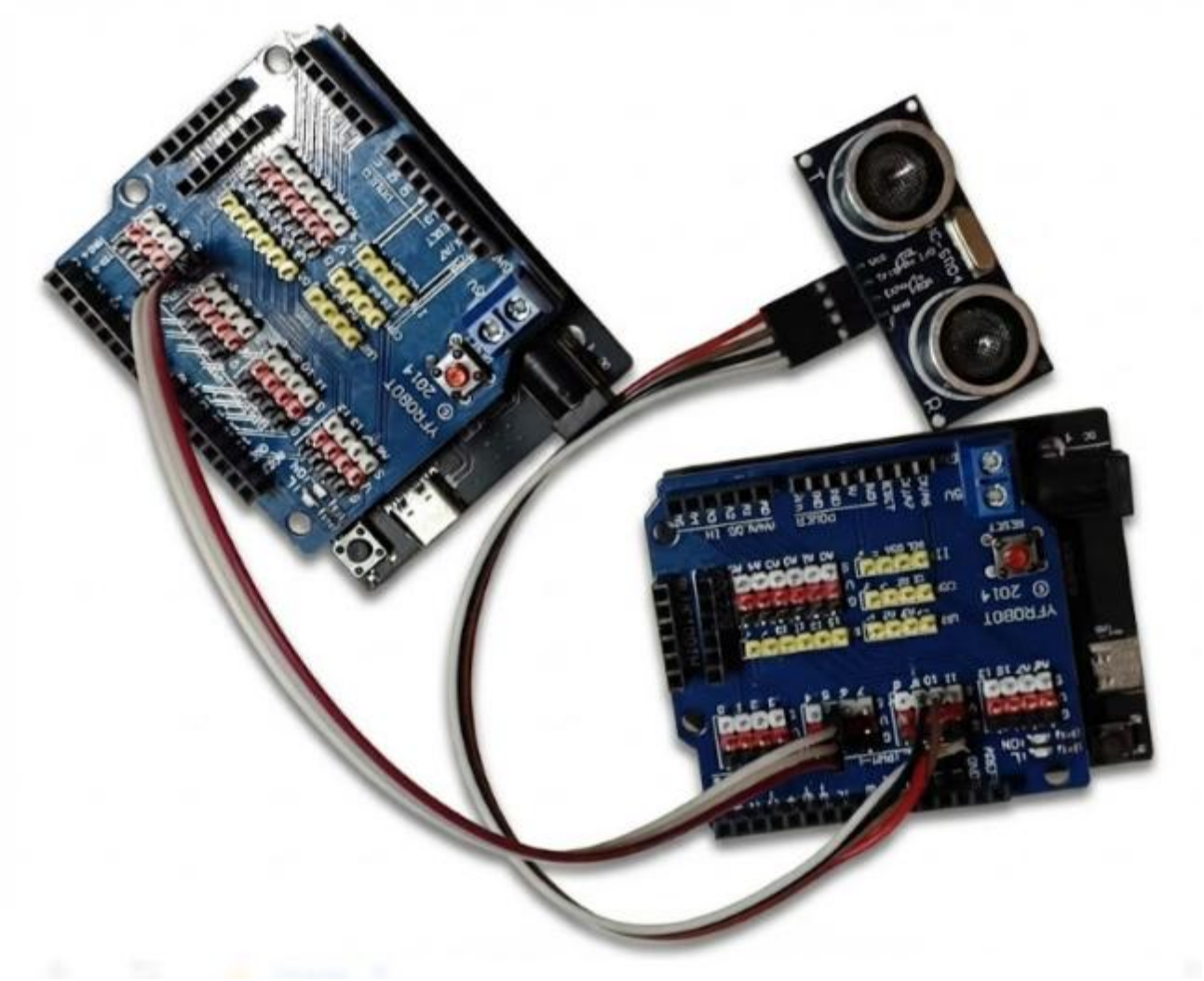

图 16 超声波测距实验实物图

2、 代码分析

(1) 硬件资源配置与全局变量初始化

程序起始部分通过常量定义了超声波模块（Trig/Echo）、伺服电机及板载 LED 的 GPIO 引脚映射，并设定了测距所需的声速常数（0.034 cm/us）。为了实现 10Hz 的高频采样任务，系统预设了 transmitInterval 为 100ms，并引入 dataCount 计数器用于追踪数据封包，确保每采集 10 组距离数据后精准触发一次“cont:10”的状态标识上报。同时，针对 SG90 舵机配置了 16 位分辨率的 PWM 硬件通道，以保障角度控制的精细度。

(2) 系统环境初始化和 PWM 引擎启动

在 setup()函数中，系统完成了硬件外设的底层初始化：首先激活测距仪专用的电源引脚并执行稳压延迟，确保传感器的物理工作环境稳定。随后，通过 ledcSetup 指令配置了 50Hz 的舵机基准频率，利用 ledcAttachPin 将信号逻辑绑定至 19 号引脚，并初始化舵机角度至 0° 位置。此外，开启波特率为 115200 的串口通讯链路，为发射端与接收端之间的高速异步数据传输构建物理接口。

(3) 非阻塞式测距算法与多阶反馈逻辑

主循环逻辑彻底摒弃了阻塞型的 delay()函数，转而采用基于 millis()的非阻塞时间戳比对机制。系统每隔 100ms 自动触发 measureDistance()函数，该函数通过向 Trig 引脚投射

10us 的触发脉冲，利用 pulseIn 捕获飞行时间（ToF）并代入声速公式计算物理距离。当探测值低于 30cm 阈值时，程序立即驱动舵机转向 90° 执行机构避障或报警动作。这种设计保证了系统在处理繁琐的数学运算时，依然能维持极高的环境感知灵敏度。

(4) 基于 map 算法的动态视觉反馈与数据封包控制

程序引入了高度智能化的“距离-频率”线性映射逻辑：通过 map()函数将实时采集的距离数值映射至[100ms, 2000ms]的时间区间内，从而动态重置 LED 的闪烁间隔 ledInterval。这意味着当障碍物靠近时，指示灯会显著提高跳闪频率，实现直观的距离预警。在通讯输出层面，系统每完成一次测距即通过串口下发距离字符串，并在 dataCount 累计至 10 次时自动补发同步标识符，配合接收端的 readStringUntil('\n')机制，确保了跨板数据流的有序解析与逻辑对齐。

3、 源代码

测距与通信实验——发送端代码

```
/*
  ESP32 每 10 个距离数据输出一次 cont:10
  功能：
  1. 每秒采集 10 个距离数据（每 100ms 一次）
  2. 每累计 10 个距离数据，输出一次"cont:10"
  3. 同步控制舵机，保留每个距离数据输出
  4. 板载 LED 随距离变化改变闪烁频率（距离越远，闪烁越慢）
*/

// 引脚定义
const int powerPin = 23; // 测距仪电源
const int trigPin = 26;  // Trig
const int echoPin = 25;  // Echo
const int servoPin = 19; // 舵机信号
const int ledPin = 2;    // 板载 LED（ESP32 默认板载 LED 为 GPIO2）

// PWM 配置（舵机）
const int servoChannel = 0;
const int servoFreq = 50;
const int servoResolution = 16;

// 数据传输参数（10Hz = 100ms 间隔）
const unsigned long transmitInterval = 100; // 传输间隔（毫秒）
unsigned long previousTransmitTime = 0;     // 上一次传输时间

// 计数参数：每 10 个数据输出一次 cont:10
const int countThreshold = 10; // 计数阈值
int dataCount = 0;             // 数据计数器

// 测距参数
const float soundSpeed = 0.034;
const long timeout = 30000;
const int threshold = 30; // 舵机动作阈值（厘米）
```

```
// LED 闪烁参数
unsigned long previousLedTime = 0; // 上一次 LED 状态切换时间
bool ledState = LOW;               // 当前 LED 状态
int ledInterval = 500;             // LED 闪烁间隔（毫秒，默认 500ms）
const int minInterval = 100;       // 最小闪烁间隔（最近距离时，100ms）
const int maxInterval = 2000;      // 最大闪烁间隔（最远距离时，2000ms）
const int maxDistance = 100;       // 最大有效距离（超过此值按最大距离处理）

void setup() {
    Serial.begin(115200);

    // 初始化测距仪电源
    pinMode(powerPin, OUTPUT);
    digitalWrite(powerPin, HIGH);
    delay(100);

    // 初始化 Trig/Echo 引脚
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);

    // 初始化舵机 PWM
    ledcSetup(servoChannel, servoFreq, servoResolution);
    ledcAttachPin(servoPin, servoChannel);
    setServoAngle(0);

    // 初始化板载 LED
    pinMode(ledPin, OUTPUT);
    digitalWrite(ledPin, LOW);

    Serial.println("开始采集距离数据，每 10 个输出一次 cont:10...");
}

void loop() {
    // 非阻塞计时：每 100ms 采集一次数据
    unsigned long currentTime = millis();
    if (currentTime - previousTransmitTime >= transmitInterval) {
        previousTransmitTime = currentTime;

        // 1. 测量距离
        float distance = measureDistance();

        // 2. 输出当前距离数据
        if (distance == -1) {
            Serial.println("距离：超出范围");
            // 超出范围时按最大距离处理 LED 闪烁
```

```
            distance = maxDistance;
        } else {
            Serial.print("距离：");
            Serial.println(distance);
            // 限制距离在有效范围内（避免过远导致闪烁间隔异常）
            distance = constrain(distance, 0, maxDistance);
        }

        // 3. 计算LED闪烁间隔（距离越远，间隔越大）
        // 映射距离到闪烁间隔：距离 0→minInterval，距离 maxDistance→
maxInterval
        ledInterval = map(distance, 0, maxDistance, minInterval,
maxInterval);

        // 4. 计数+判断是否达到10个数据
        dataCount++;
        if (dataCount >= countThreshold) {
            Serial.println("cont:10"); // 输出标识
            dataCount = 0;             // 重置计数器
        }

        // 5. 控制舵机
        if (distance <= threshold && distance != -1) {
            setServoAngle(90);
        } else {
            setServoAngle(0);
        }
    }

    // 非阻塞控制LED闪烁（根据计算的间隔切换状态）
    if (currentTime - previousLedTime >= ledInterval) {
        previousLedTime = currentTime;
        ledState = !ledState; // 切换LED状态（亮→暗或暗→亮）
        digitalWrite(ledPin, ledState);
    }
}

// 测量距离函数（返回厘米，无效返回-1）
float measureDistance() {
    long duration;
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
```

```
    duration = pulseIn(echoPin, HIGH, timeout);

    if (duration == 0) {
        return -1; // 超出范围返回无效值
    } else {
        return duration * soundSpeed / 2;
    }
}

// 舵机角度控制函数
void setServoAngle(int angle) {
    angle = constrain(angle, 0, 180);
    int us = map(angle, 0, 180, 500, 2500);
    uint32_t duty = (us * (1 << servoResolution)) / 20000;
    ledcWrite(servoChannel, duty);
}
```

测距与通信实验——接收端代码

```
/*
  接收板：接收发送板的数据并输出
  功能：通过 RX 引脚读取数据，原样输出到串口监视器
*/

void setup() {
    Serial.begin(115200); // 波特率必须和发送板一致（115200）
    while (!Serial) {
    } // 等待串口就绪
    Serial.println("接收板启动：等待接收数据...");
}

void loop() {
    // 若收到数据，立即读取并输出
    if (Serial.available() > 0) {
        String data = Serial.readStringUntil('\n'); // 读取一行数据
        Serial.println(data);                       // 输出到接收板的串口监
视器
    }
}
```

4、 小结

本次“测距与通讯实验”的成功开展，使我深度掌握了超声波测距原理及跨板串口异步通讯的核心技术。在实验过程中，我通过编写发射端代码，实现了利用 HC-SR04 模组进行高精度距离捕获，并配合硬件定时器中断与任务调度机制，达成了每秒 10 次的稳定数据上报

频率。同时，系统集成的动态视觉反馈逻辑——即 LED 闪烁频率随测距数值灵敏变化的特性，进一步验证了程序在处理实时数据流时的可靠性。尽管在硬件联调初期，由于拓展板引脚分布较为密集，我遭遇了因串口 TX/RX 引脚反接导致的通讯瘫痪问题，但通过对照端口映射表进行逐一排查，最终成功定位并解决了故障。这次经历让我深刻领悟到，在处理复杂的分布式单片机系统时，维持清晰的接线记录与严谨的功能定义是提升实验效率、规避工程隐患的关键。通过这一阶段的实践，我不仅提升了对底层时序控制的理解，更在解决实际通讯冲突的过程中积累了宝贵的硬件调试经验，为后续开发更为复杂的物联网集成系统奠定了坚实基础。

四、物联网——远程控制小灯实验

1、 题目描述与分析

(1) 题目描述

本实验旨在构建一套基于云端同步技术的物联网远程控制系统，实现通过移动终端对 ESP32 开发板板载 LED 状态的跨网络精准调度。实验要求开发者在程序底层嵌入移动热点的 SSID 与加密凭证（密码），通过无线网络链路将主控终端接入互联网。

具体要求：

① 在固件中集成网络配置协议并完成烧录；

② 通过开启手机热点建立本地接入网关，确保 ESP32 能够与“点灯科技”云端服务器成功握手；

③ 利用专用 App 界面构建交互控件，实现对硬件引脚电平状态的远程触发与实时监控。

(2) 点灯科技平台介绍

本实验采用了“点灯科技（Blinker）”这一专业级的物联网全栈解决方案作为逻辑支撑平台。该平台深度整合了高性能云端服务器、跨平台移动应用（App）以及完善的设备端 SDK 支撑体系，大幅降低了嵌入式设备接入互联网的门槛。依托其提供的分布式通信架构，开发者可以迅速将 ESP32 开发板转化为可联网的智能终端。通过移动端终端，用户不仅能够对远端硬件下发控制逻辑，还能实时调取设备的运行参数与在线状态，从而构建起稳定、低延迟的硬件交互闭环。

图 17 点灯科技

(3) 题目分析

远程控制小灯实验的核心逻辑在于跨网络层次的指令透传。系统必须在物理层（硬件连接）、网络层（WiFi 协议栈配置）与应用层（App 控件交互）之间建立可靠的逻辑映射。从工程实现角度分析，本项目的关键难点在于 ESP32 的网络环境适配与云端身份认证。系统需要处理复杂的网络配置逻辑，确保主控板在热点覆盖范围内能够自动执行重连机制。同时，通过“点灯科技”App 发出的操作指令需经由云端服务器中转并精准映射至硬件对应的 GPIO 引脚。这种基于云端的命令转发机制，结合移动热点的便携式组网方案，完整演示了现代物联网应用中“App-云-端”的典型架构模型，为后续开发更复杂的智慧家居系统奠定了实践基础。

2、 代码分析

本程序基于点灯科技（Blinker）协议栈构建，核心逻辑围绕“网络认证、事件绑定、指令分发”三个维度展开。在初始化阶段，系统通过配置唯一的 auth 认证令牌与 WiFi 凭证，使 ESP32 开发板能够跨越局域网与云端服务器建立稳定的加密连接。技术实现层面，程序采用了高效的事件驱动架构：利用 Button1.attach()将移动端 App 控件与 button1_callback 异步回调函数进行逻辑绑定；每当云端指令透传至硬件端时，系统即刻触发回调逻辑，通过 digitalWrite 指令实现对 GPIO 物理引脚状态的精准反转。在 loop() 主循环中，系统仅依靠非阻塞的 Blinker.run()指令维持心跳连接与实时数据轮询，确保了在极低系统开销下，依然能实现毫秒级的远程交互响应。

3、 源代码

远程控制小灯实验

```
#define BLINKER_WIFI

#include <Blinker.h>

char auth[] = "2d54da4c8580";
char ssid[] = "Redmi K70";
char pswd[] = "sdsntwmxhn";

// 新建组件对象
BlinkerButton Button1("btn-abc");
BlinkerNumber Number1("num-abc");

int counter = 0;

// 按下按键即会执行该函数
void button1_callback(const String& state) {
    BLINKER_LOG("get button state: ", state);
    digitalWrite(LED_BUILTIN, !digitalRead(LED_BUILTIN));
}

// 如果未绑定的组件被触发，则会执行其中内容
void dataRead(const String& data) {
    BLINKER_LOG("Blinker readString: ", data);
    counter++;
    Number1.print(counter);
```

```
}

void setup() {
    // 初始化串口
    Serial.begin(115200);
    BLINKER_DEBUG.stream(Serial);
    BLINKER_DEBUG.debugAll();

    // 初始化有 LED 的 IO
    pinMode(LED_BUILTIN, OUTPUT);
    digitalWrite(LED_BUILTIN, HIGH);
    // 初始化 blinker
    Blinker.begin(auth, ssid, pswd);
    Blinker.attachData(dataRead);

    Button1.attach(button1_callback);
}

void loop() { Blinker.run(); }
```

4、 小结

图 18 远程控制小灯实验图

本次“远程控制小灯实验”是迈向物联网（IoT）开发领域的关键技术实践，其核心目标在于构建移动端与ESP32主控板之间跨网络的实时通讯链路。通过配置“点灯科技(Blinker)”App 界面并完成 ESP32 的 WiFi 接入逻辑，我成功实现了利用手机终端对硬件端 LED 亮灭状态

的远程精确调度。在实验过程中，我深入掌握了单片机的网络配置流程、设备在线状态维持及异步指令解析等关键环节，深刻体会到物联网技术作为万物互联核心的强大功能与应用前景。此次实践不仅显著提升了我的 WiFi 通讯编程能力，更在系统集成与人机交互层面为后续“挑战任务”中更为复杂的蓝牙遥控小车项目奠定了坚实的底层逻辑与实操基础。

五、小车组装与测试实验

1、 硬件组装与测试

（1） 题目描述

将小车控制代码烧写至 ESP32 开发主控板，并完成小车底盘机械结构、动力电池组及核心控制模块的物理组装与电路接驳。确保小车在无人干预的自动化模式下，能够严格遵循“直线行驶—向左旋转—后退—向右旋转”的预设时序逻辑执行周期性闭环运动循环。

具体要求：

① 将代码烧写进代码；

② 组装小车，连接电路，按照开发板；

③ 小车自动按照流程循环运行：“行进、转向、后退转向”；

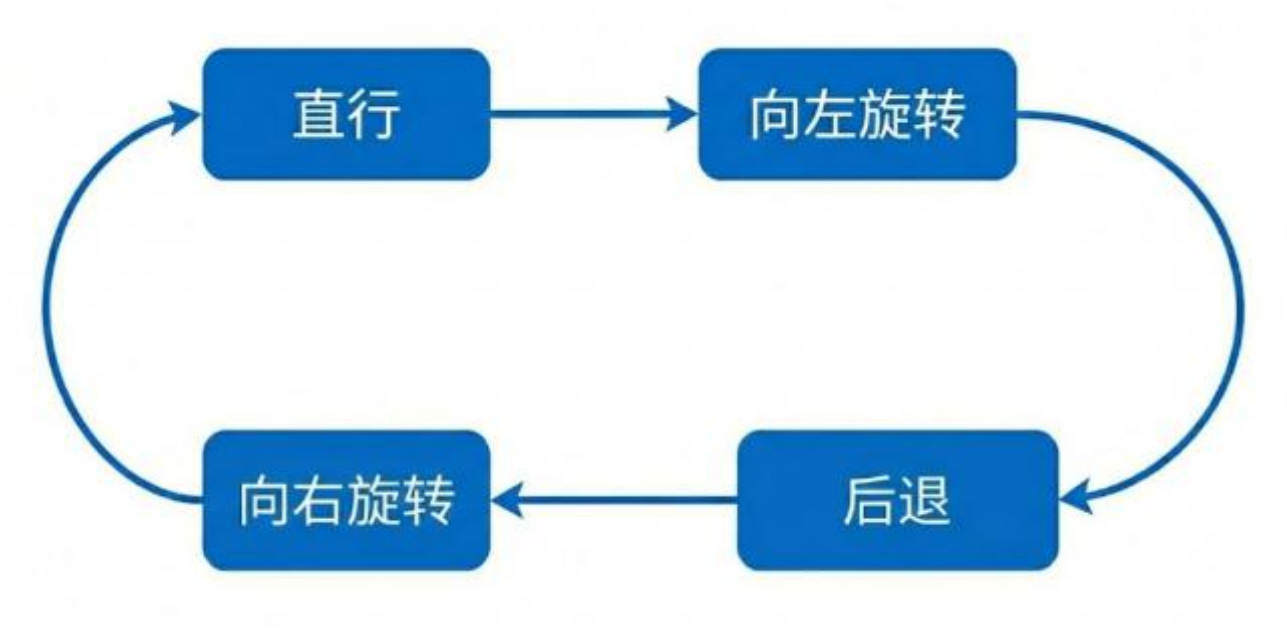

图 19 小车运动流程图

（2） 硬件设备介绍

本实验以铝合金车架作为物理承载平台，核心控制单元由 ESP32 开发板与 R3-L293D 电机驱动扩展板耦合构成，另外还有一对轮胎和霍尔编码电机。

① R3-L293D 驱动扩展板

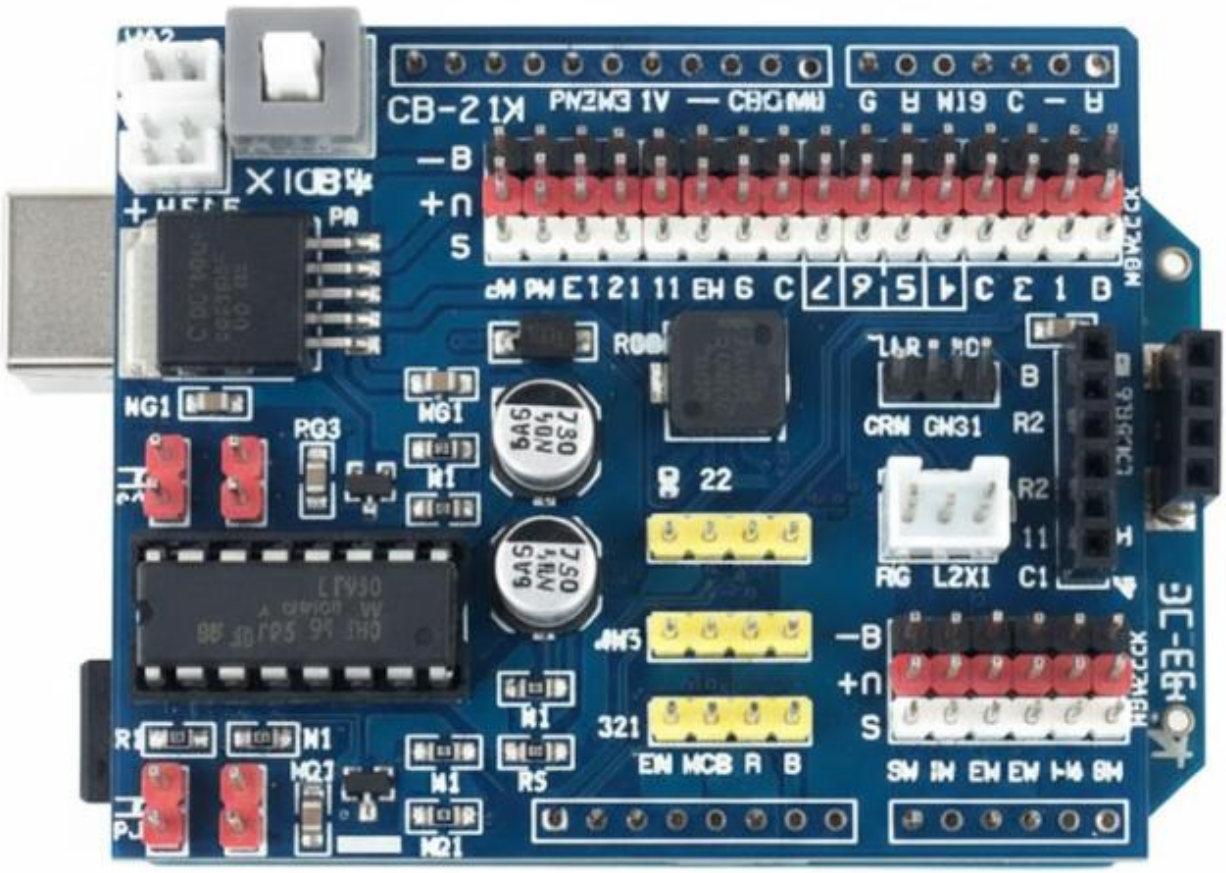

图 20 R3L293D 扩展板

R3-L293D 驱动扩展是本实验小车系统的电力调度与运动控制中枢，R3-L293D 驱动扩展板采用了高性能的 H 桥集成电路架构，能够完美兼容 ESP32 主控单元的逻辑信号输出。此外，该扩展板采用了标准化的 Arduino R3 接口布局，并额外引出了丰富的 GPIO 排针与 PH2.0 专用接口，大幅降低了电机接线的复杂度并提升了电气连接的物理强度。该驱动模块的具体性能规格详见表 10。

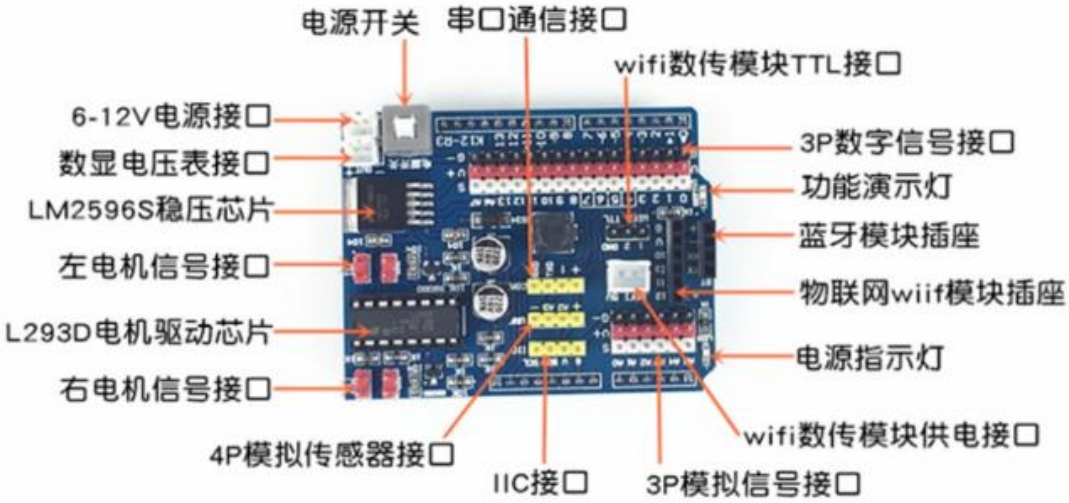

图 21 扩展板接口

表 10 R3-L293D 驱动扩展板参数表

参数	数值
输入电压	6V12V
输出电压	5V
电机驱动	L293D H 桥路
稳压芯片	LM2596S
引脚说明	所有 I/O 口全部排针引出
电机接口	2.54 间距排针，可使用仕邦线连接
特殊说明	PH2.0 插头线，供外接电源和电压表使用
适用范围	适用于 Arduino UNO 开发板

② 霍尔编码电机

图 22 霍尔编码电机

霍尔感应器直流减速电机，实现了驱动与测速的一体化。该电机在尾部加装了磁性码盘，通过霍尔元件捕获旋转产生的磁通量变化，从而将其转化为标准的电脉冲信号。

表 11 霍尔编码电机引脚定义

引脚标号	说明
G	霍尔供电负极
H_1	霍尔 H_1 输出信号，方波
H_1	霍尔 H_2 输出信号，方波
V	霍尔供电正极
M+	电机供电正极
M-	电机供电负极

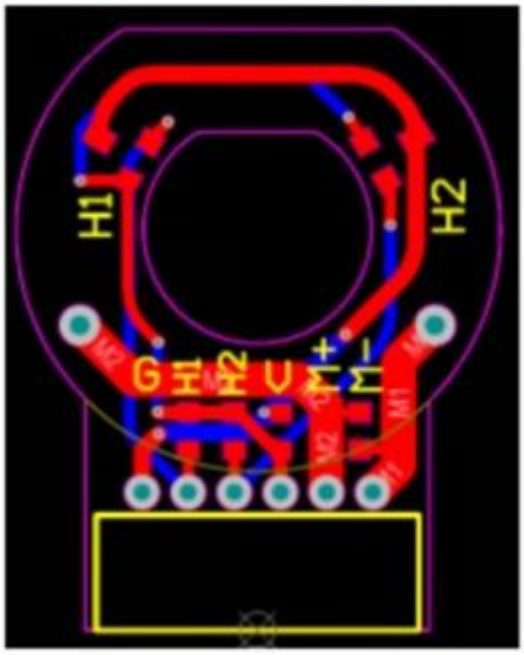

图 23 霍尔码电机电路板布局

霍尔编码电机工作原理：

1. 电机转速测量：霍尔传感器可以感应磁性码盘的 N/S 极变换。当传感器感应到 S 极时输出高电平，感应到 N 极时输出低电平。电机连续转动时，霍尔引脚将输出周期性的方波信号。通过控制器测量方波的周期 T，并已知码盘的脉冲对数为 P，即可利用以下公式计算电机的实时转速 n：

$$n = \frac{60}{PT}$$

2. 电机正反转测量：电机内部在不同空间位置放置了两个霍尔感应器（H1 与 H2）。当电机正转或反转时，两路信号的相位偏移顺序不同。

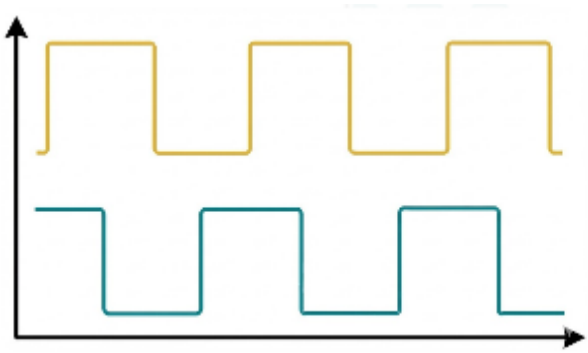

图 24 电机正转

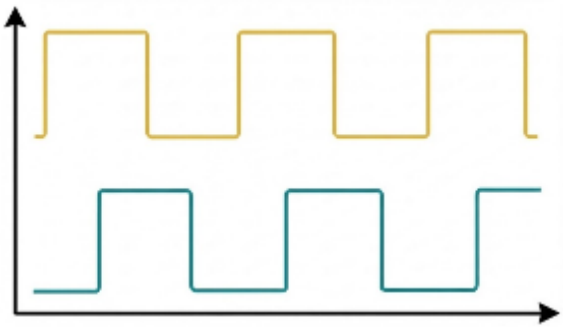

图 25 电机反转

如图 24 与图 25 所示，电机正转时，H1 信号的上升沿领先于 H2；而电机反转时，H1 信号的下降沿落后于 H2。通过判断两路方波的相位差，主控系统即可精准识别小车的当前行进方向。

（3） 硬件组装

本实验的硬件组装遵循“先机械后电气”的模块化整合原则。首先紧固 65mm 规格的橡胶轮胎，确保动力执行机构机械连接稳固且运行顺畅，无结构干涉。随后，将 R3-L293D 驱动扩展板与 ESP32 开发板进行垂直堆叠安装，操作时需确保排针对位精准，严防因受力不均导致针脚弯曲损坏。

在电气接线阶段，将左右电机的动力控制线接入驱动板的接线端子，并按照引脚定义将霍尔编码器的 H1、H2 信号线与电源线正确接入扩展板的信号采集口。最后，将动力电池组连接至电源输入端口，重点核对 6V～12V 供电的正负极极性，确保系统电位基准一致。组装完成后，需通过全方位视觉检查确认导线布局整齐且无短路隐患，即可构建起完整的小车。

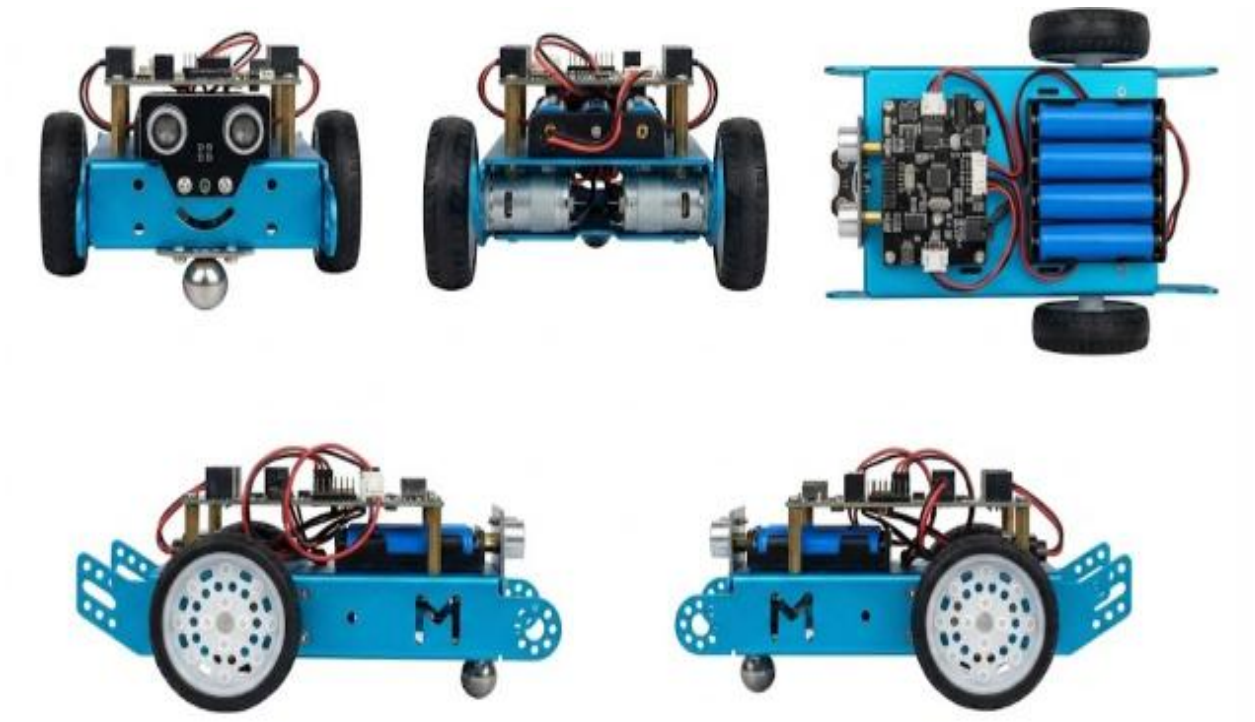

图 26 小车各视角视图

2、 代码分析

本实验程序基于 ESP32 的双核架构开发，通过硬件中断与多任务调度实现了对电机转速的精确闭环控制。在数据采集层，程序利用外部中断实时捕获编码器输出的边沿信号，并根据引脚电平逻辑判定电机的旋转方向。系统采用典型的 M 法测速原理，通过定时器每隔 20 毫秒触发一次数据读取函数，计算该周期内的脉冲增量，从而实时转化为小车的瞬时反馈速度值。这种高频采样机制为后续的动力补偿提供了准确的数据基础。

运动控制核心逻辑由 FreeRTOS 任务驱动，并将其固定在处理器的特定核心运行，以确保控制指令不受其他进程干扰，保障了操控的实时稳定性。在算法设计上，程序引入了启动补偿与增量式调节机制。针对直流电机的物理特性，系统预设了启动初始量，辅助电机迅速克服静摩擦力。在巡航阶段，控制算法会实时比对目标设定速度与编码器反馈的实际速度，根据两者的偏差值，结合预设的步进限幅和平滑因子，动态调整输出到电机的驱动信号占空比。

3、 源代码

小车组装与测试实验

```
#define ENCODER_L 12       // 8 号引脚
#define DIRECTION_L 13     // 9 号引脚
#define ENCODER_R 25       // 3 号引脚
#define DIRECTION_R 26     // 2 号引脚
#define interrupt_time 20 // 中断时间
#define L_DIR 14           // 7 号引脚
#define L_SPEED 16         // 5 号引脚
#define R_DIR 17           // 4 号引脚
#define R_SPEED 27         // 6 号引脚

#define TARGET_L 1500.0
#define TARGET_R 1300.0
// ALPHA（建议在 100-500 之间）值越小和 MAXINC（建议不越过 10）越大，速度调节越快，太快容易转速调节过头
#define ALPHA 300.0
#define MAXINC 8.0
// 轮子启动初始量，使轮子快束起动
#define BOOST 80.0

#include <Arduino.h>
#include <Ticker.h>
Ticker timer_read_encoder; // 中断函数定时器定义

int32_t Velocity_L, Velocity_R;        // 左右轮编码器数据
int16_t Velocity_Left, Velocity_Right; // 左右轮速度
void READ_ENCODER_L(void);
void READ_ENCODER_R(void);
void read(void);

void READ_ENCODER_L(void) {
    if (digitalRead(ENCODER_L) == LOW) {
```

```
        // 如果是下降沿触发的中断
        if (digitalRead(DIRECTION_L) == LOW)
            Velocity_L--;
        // 根据另外一相电平判定方向
        else
            Velocity_L++;
    } else { // 如果是上升沿触发的中断
        if (digitalRead(DIRECTION_L) == LOW)
            Velocity_L++; // 根据另外一相电平判定方向
        else
            Velocity_L--;
    }
}
void READ_ENCODER_R(void) {
    if (digitalRead(ENCODER_R) == LOW) { // 如果是下降沿触发的中断
        if (digitalRead(DIRECTION_R) == LOW)
            Velocity_R--; // 根据另外一相电平判定方向
        else
            Velocity_R++;
    } else { // 如果是上升沿触发的中断
        if (digitalRead(DIRECTION_R) == LOW)
            Velocity_R++; // 根据另外一相电平判定方向
        else
            Velocity_R--;
    }
}
// void read(void) {
////  Velocity_Left = Velocity_L * 2000 / interrupt_time;
////  Velocity_L = 0;
/////读取左轮编码器数据，并清零，这就是通过M法测速（单位时间内的脉冲数）得到速度。
////  Velocity_Right = Velocity_R * 2000 / interrupt_time;
////  Velocity_R = 0; //读取右轮编码器数据，并清零
//}

void read(void) {
    noInterrupts(); // 关闭所有中断
    Velocity_Left = Velocity_L * 2000 / interrupt_time;
    Velocity_L = 0; // 读取左轮编码器数据，并清零
    Velocity_Right = Velocity_R * 2000 / interrupt_time;
    Velocity_R = 0; // 读取右轮编码器数据，并清零
    interrupts();   // 重新开启中断
}

float l_speed = 0, r_speed = 0;
void taskOne(void* pvParameters) {
```

```
    int i;
    while (1) {
        l_speed = 0;
        r_speed = 0;
        for (i = 0; i < 65; i++) {
            digitalWrite(L_DIR, 1);
            digitalWrite(R_DIR, 1);
            analogWrite(L_SPEED, l_speed);
            analogWrite(R_SPEED, r_speed);
            if (i == 10) {
                l_speed = BOOST;
                r_speed = BOOST;
            } else if (i > 10) {
                if (Velocity_Left < (TARGET_L - 50))
                    l_speed =
                        l_speed + ((TARGET_L - Velocity_Left) / ALPHA > MAXINC
                                        ? MAXINC
                                        : (TARGET_L - Velocity_Left) / ALPHA);
                else if (Velocity_Left > (TARGET_L + 50))
                    l_speed =
                        l_speed - ((Velocity_Left - TARGET_L) / ALPHA > MAXINC
                                        ? MAXINC
                                        : (Velocity_Left - TARGET_L) / ALPHA);
                if (Velocity_Right < (TARGET_R - 50))
                    r_speed =
                        r_speed + ((TARGET_R - Velocity_Right) / ALPHA > MAXINC
                                        ? MAXINC
                                        : (TARGET_R - Velocity_Right) / ALPHA);
                else if (Velocity_Right > (TARGET_R + 50))
                    r_speed =
                        r_speed - ((Velocity_Right - TARGET_R) / ALPHA > MAXINC
                                        ? MAXINC
                                        : (Velocity_Right - TARGET_R) / ALPHA);
                if (l_speed > 255)
                    l_speed = 255;
                if (r_speed > 255)
                    r_speed = 255;
            }
            delay(50);
        }

        l_speed = 0;
        r_speed = 0;
        for (i = 0; i < 65; i++) {
            digitalWrite(L_DIR, 0);
            digitalWrite(R_DIR, 1);
```

```
        analogWrite(L_SPEED, l_speed);
        analogWrite(R_SPEED, r_speed);
        if (i == 10) {
            l_speed = BOOST;
            r_speed = BOOST;
        } else if (i > 10) {
            if (Velocity_Left > -(TARGET_L - 50))
                l_speed =
                    l_speed + ((Velocity_Left + TARGET_L) / ALPHA > MAXINC
                                    ? MAXINC
                                    : (Velocity_Left + TARGET_L) / ALPHA);
            else if (Velocity_Left < -(TARGET_L + 50))
                l_speed =
                    l_speed - ((-TARGET_L - Velocity_Left) / ALPHA > MAXINC
                                    ? MAXINC
                                    : (-TARGET_L - Velocity_Left) / ALPHA);
            if (Velocity_Right < (TARGET_R - 50))
                r_speed =
                    r_speed + ((TARGET_R - Velocity_Right) / ALPHA > MAXINC
                                    ? MAXINC
                                    : (TARGET_R - Velocity_Right) / ALPHA);
            else if (Velocity_Right > (TARGET_R + 50))
                r_speed =
                    r_speed - ((Velocity_Right - TARGET_R) / ALPHA > MAXINC
                                    ? MAXINC
                                    : (Velocity_Right - TARGET_R) / ALPHA);
            if (l_speed > 255)
                l_speed = 255;
            if (r_speed > 255)
                r_speed = 255;
        }
        delay(50);
    }
    l_speed = 0;
    r_speed = 0;
    for (i = 0; i < 65; i++) {
        digitalWrite(L_DIR, 0);
        digitalWrite(R_DIR, 0);
        analogWrite(L_SPEED, l_speed);
        analogWrite(R_SPEED, r_speed);
        if (i == 10) {
            l_speed = BOOST;
            r_speed = BOOST;
        } else if (i > 10) {
            if (Velocity_Left > -(TARGET_L - 50))
                l_speed =
```

```
                l_speed + ((Velocity_Left + TARGET_L) / ALPHA > MAXINC
                                ? MAXINC
                                : (Velocity_Left + TARGET_L) / ALPHA);
            else if (Velocity_Left < -(TARGET_L + 50))
                l_speed =
                    l_speed - ((-TARGET_L - Velocity_Left) / ALPHA > MAXINC
                                ? MAXINC
                                : (-TARGET_L - Velocity_Left) / ALPHA);
            if (Velocity_Right > -(TARGET_R - 50))
                r_speed =
                    r_speed + ((Velocity_Right + TARGET_R) / ALPHA > MAXINC
                                ? MAXINC
                                : (Velocity_Right + TARGET_R) / ALPHA);
            else if (Velocity_Right < -(TARGET_R + 50))
                r_speed =
                    r_speed - ((-TARGET_R - Velocity_Right) / ALPHA > MAXINC
                                ? MAXINC
                                : (-TARGET_R - Velocity_Right) / ALPHA);
            if (l_speed > 255)
                l_speed = 255;
            if (r_speed > 255)
                r_speed = 255;
        }
        delay(50);
    }
    l_speed = 0;
    r_speed = 0;
    for (i = 0; i < 65; i++) {
        digitalWrite(L_DIR, 1);
        digitalWrite(R_DIR, 0);
        analogWrite(L_SPEED, l_speed);
        analogWrite(R_SPEED, r_speed);
        if (i == 10) {
            l_speed = BOOST;
            r_speed = BOOST;
        } else if (i > 10) {
            if (Velocity_Left < (TARGET_L - 50))
                l_speed =
                    l_speed + ((TARGET_L - Velocity_Left) / ALPHA > MAXINC
                                ? MAXINC
                                : (TARGET_L - Velocity_Left) / ALPHA);
            else if (Velocity_Left > (TARGET_L + 50))
                l_speed =
                    l_speed - ((Velocity_Left - TARGET_L) / ALPHA > MAXINC
                                ? MAXINC
                                : (Velocity_Left - TARGET_L) / ALPHA);
```

```
                if (Velocity_Right > -(TARGET_R - 50))
                    r_speed =
                        r_speed + ((Velocity_Right + TARGET_R) / ALPHA > MAXINC
                                        ? MAXINC
                                        : (Velocity_Right + TARGET_R) / ALPHA);
                else if (Velocity_Right < -(TARGET_R + 50))
                    r_speed =
                        r_speed - ((-TARGET_R - Velocity_Right) / ALPHA > MAXINC
                                        ? MAXINC
                                        : (-TARGET_R - Velocity_Right) / ALPHA);
                if (l_speed > 255)
                    l_speed = 255;
                if (r_speed > 255)
                    r_speed = 255;
            }
            delay(50);
        }
    }
}

void setup() {
    Serial.begin(115200);
    pinMode(ENCODER_L, INPUT);   // 编码器引脚 输入模式
    pinMode(ENCODER_R, INPUT);   // 编码器引脚 输入模式
    pinMode(DIRECTION_L, INPUT); // 编码器引脚 输入模式
    pinMode(DIRECTION_R, INPUT); // 编码器引脚 输入模式
    pinMode(L_DIR, OUTPUT);
    pinMode(L_SPEED, OUTPUT);
    pinMode(R_DIR, OUTPUT);
    pinMode(R_SPEED, OUTPUT);
    // 编码器接口 1 开启外部跳边沿中断
    attachInterrupt(ENCODER_L, READ_ENCODER_L, CHANGE);
    // 中断函数 READ_ENCODER_L
    // 编码器接口 2 开启外部跳边沿中断
    attachInterrupt(ENCODER_R, READ_ENCODER_R, CHANGE);
    // 中断函数 READ_ENCODER_R
    interrupts();
    // 打开外部中断
    timer_read_encoder.attach_ms(interrupt_time, read);

    xTaskCreatePinnedToCore(taskOne,    /* Task function. */
                            "TaskOne", /* String with name of task. */
                            1000,      /* Stack size in bytes. */
                            NULL, /* Parameter passed as input of the task */
                            1,    /* Priority of the task. */
                            NULL, /* Task handle. */
```

```
                                1);
}

void loop() {
    Serial.print("Velocity_L:");
    // Serial.print(Velocity_L);
    Serial.print(l_speed);
    Serial.print(" | Velocity_Left:");
    Serial.print(Velocity_Left);
    Serial.print(" ----- Velocity_R:");
    // Serial.print(Velocity_R);
    Serial.print(r_speed);
    Serial.print(" | Velocity_Right:");
    Serial.println(Velocity_Right);
}
```

4、 小结

本次“小车组装与测试实验”是整个机器人技术课程中从理论逻辑迈向物理集成的关键转折点。虽然在操作层面主要涉及硬件组装与基础代码烧录，但其在建立移动机器人运动控制底层认知方面具有不可替代的重要性。 在实验过程中，通过对直流减速电机、驱动模块及 ESP32 主控板的电气连接，我全面解构了差速驱动机器人的机械结构与动力链路设计。

在软件调试阶段，我重点研究了如何通过调节左右两侧轮组的 PWM 占空比来实现航向的精密补偿。针对物理电机因机械摩擦偏差导致的“行驶偏移”现象，我通过反复实验找到了最优的参数平衡点，确保了小车能够维持稳定的直线巡航轨迹。这一调试过程使我深刻领悟到“软件定义硬件”的深刻含义，掌握了差速控制实现转向的核心逻辑。

六、编程小车仿真实验

1、 题目描述与分析

（1）题目描述

通过构建高精度的 C++控制算法，驱动“VirtualRobot 编程小车仿真系统”中的虚拟智能终端执行复杂的路径规划任务。系统要求在 60s 的硬性时间阈值内，主控程序需指挥小车精准识别并依次穿越 4 个离散的目标位点，同时利用环境感知逻辑实时规避随机分布的物理障碍。

具体要求：

① 在虚拟仿真平台 VirtualRobot 中完成小车动力系统与感知原件的逻辑组装与初始化配置；

② 针对随机起始点开发动态识别算法，确保小车能实时捕获 4 个目标点的空间坐标并完成全局路径规划；

③ 在运动过程中维持高频的环境扫描，自主识别并规避各类复杂障碍物，以最短时间路径完成任务；

④ 严格遵循目标点的逻辑顺序执行穿越任务，确保系统在单位时间内完成所有预设位点的物理闭环。

（2）VirtualRobot 编程小车仿真系统介绍

编程小车仿真系统是一套深度依托 Unreal Engine 5（UE5）高性能物理引擎开发的先进在线机器人实验平台。该系统通过构建高保真度的三维物理实验场景与多元化的机器人运动模型，实现了底层执行机构与上层控制算法的高度解耦，使用户能够仅通过 C++代码驱动的方式，在虚拟维度内实现对智能小车运动轨迹与感知反馈的精密调控。

相较于传统实机实验，该平台具备全天候（24 小时）在线运行的高可用特性，彻底打破了物理场地与实验时段的空间制约，支持开发者在校园网环境下随时开展大规模的算法迭代与逻辑验证。

此外，仿真系统集成了强大的实时数据捕获与日志记录功能，能够以毫秒级精度呈现小车的动态坐标与传感器参数；配合支持任意视角切换的多维可视化交互界面，用户可以直观捕捉真机实验中难以察觉的碰撞细节与逻辑死角，从而显著提升了程序调试的透明度、准确性与开发效率。

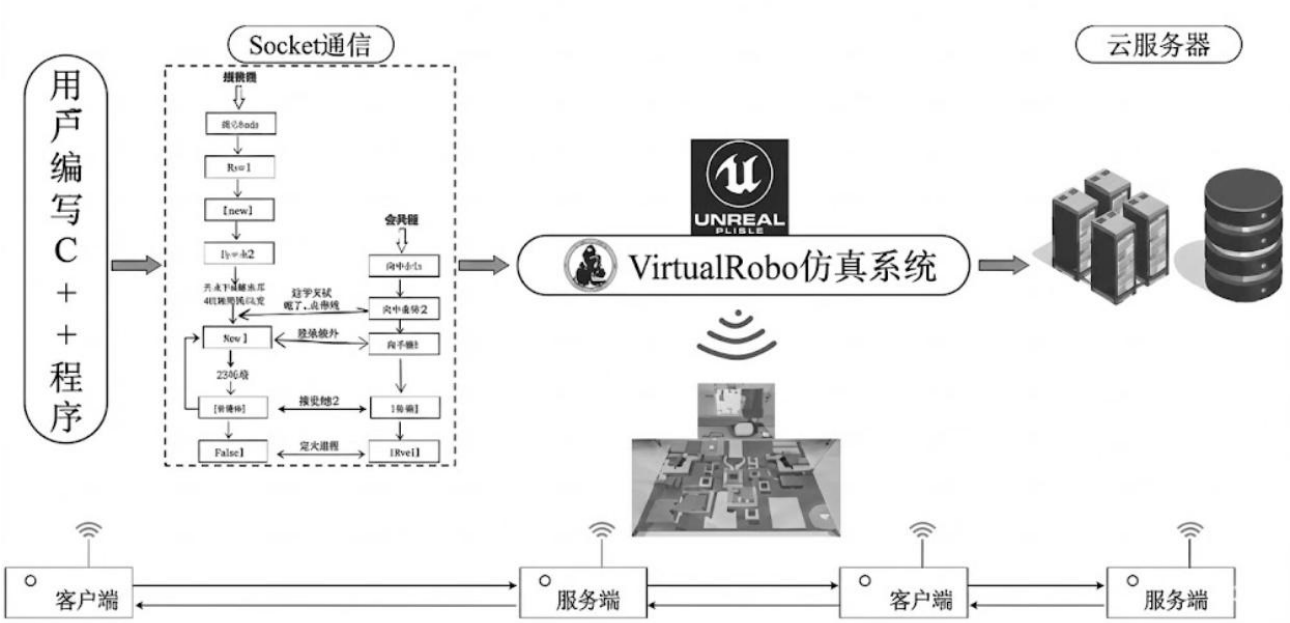

图 27 仿真系统

（3）题目分析

① 坐标与地图

小车主控程序必须实时获取并处理自身的位姿状态信息，包括当前的空间坐标、航向角（Heading）以及瞬时速度等关键物理参量。本实验采用 C++编写的控制程序，通过 Socket 异步通讯协议与基于虚幻引擎（UE）构建的仿真平台进行高频数据交互。初始化成功，仿真平台将向客户端透传完整的环境建模数据，包括地图边界、生成的物理障碍物坐标以及 4 个待穿越的目标点位信息。

为了更直观地分析运动轨迹，我们将平台发送的离散坐标绘制为如图 所示的 2D 栅格地图。图中清晰标注了小车与 4 个目标点的相对几何关系。算法设计时需重点考虑起始点与障碍物边缘的避障冗余，通过在坐标系中构建势场或搜索树，确保小车在执行“蓝、黄、粉、青”这一时序任务时，能够以最短的时间开销规避物理碰撞，从而实现最优化的仿真运行表现。

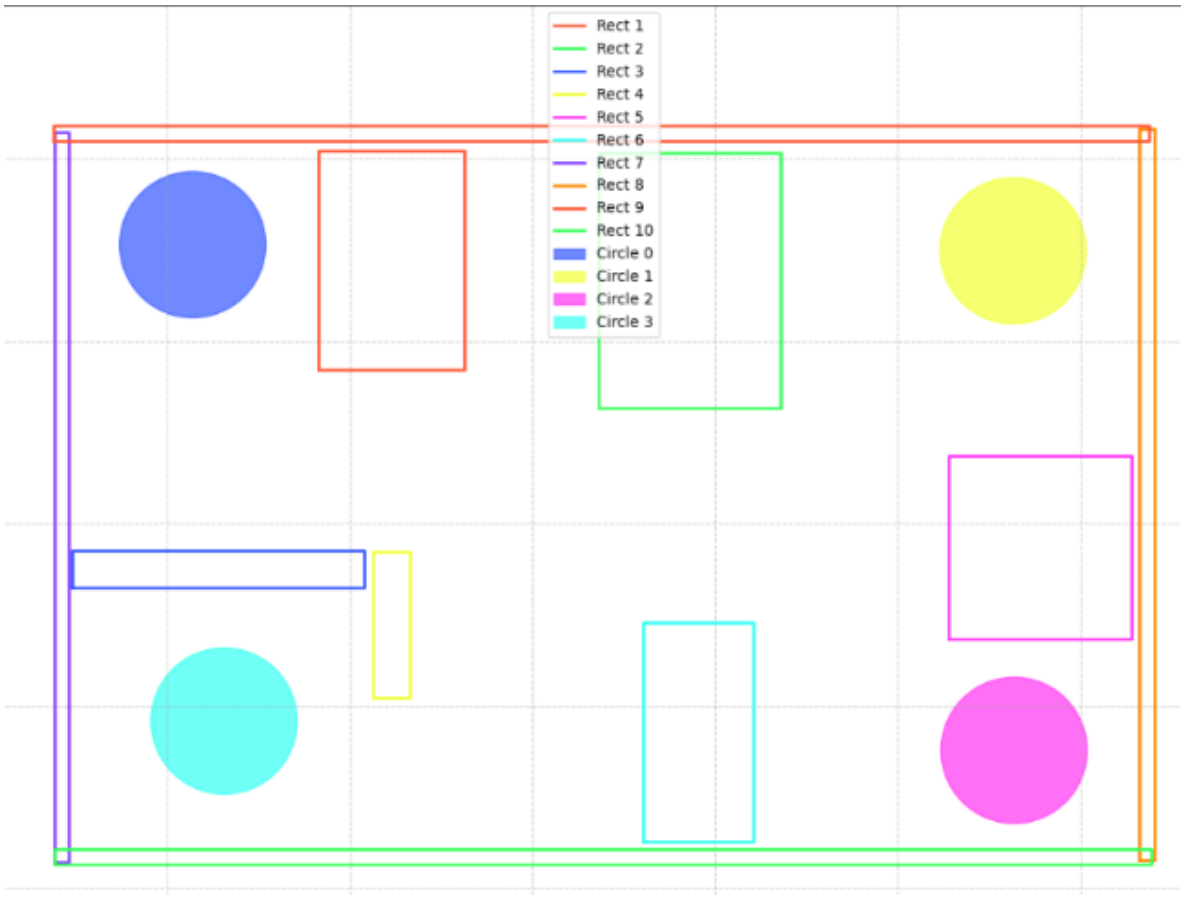

图 28 小车运动地图

② 小车控制原理

本实验采用经典的 PID（比例-积分-微分）控制算法作为小车行进与转弯的核心驱动逻辑。作为工业控制领域应用最广泛的闭环控制算法，PID 通过对设定目标值与实时反馈值之间的偏差进行线性组合计算，输出精确的控制量以驱动执行机构。

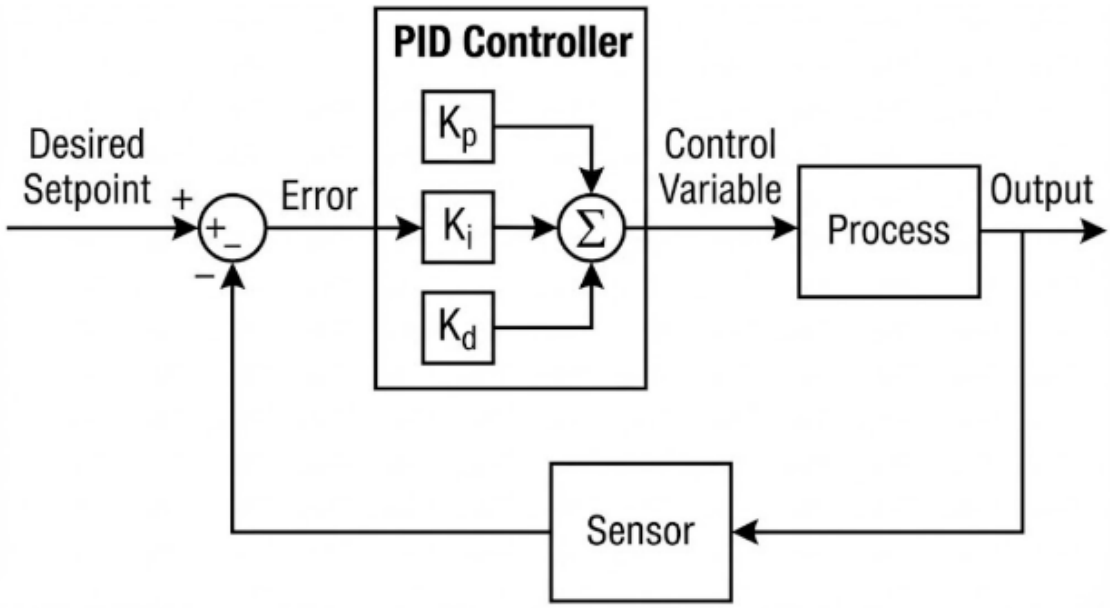

图 29 PID 控制流程图

1. PID 算法数学模型

系统的输入 r(t)为控制目标（如目标角度或距离），输出 y(t)为实际反馈值，两者的偏差定义为 e(t) = r(t) - y(t)。典型的 PID 控制函数关系式如下：

$$u(t) = K_p e(t) + K_i \int_0^t e(t)dt + K_t \frac{de(t)}{dt}$$

公式中包含三个核心调节环节：比例调节（P）负责依据偏差大小进行实时线性调节；积分调节（I）用于消除系统的稳态误差；微分调节（D）则根据偏差的变化速率预测趋势，起到阻尼作用并抑制超调。

2. 代码中的控制策略实现

在本实验的 C++ 代码中，系统针对不同运动模态对 PID 算法进行了针对性的简化应用。在转弯控制函数 turnToHeading 中，程序采用了高效的比例控制（P 控制）策略：通过计算目标航向角与当前 IMU 反馈角度的偏差 error，乘以特定的比例增益（如 SPEED 变量及其系数），动态调整左右轮的转速比。这种设计使小车在角度偏差较大时能快速旋转，而在接近目标角度时自动减速缓冲，有效防止了物理旋转过程中的震荡。

在直线行驶控制函数 moveForward 中，系统不仅利用距离偏差作为主控制量，还引入了姿态修正逻辑。代码通过 angle_error 获取航向偏差，并施加 kp_angle 比例补偿量，通过 leftSpeed = base + correction 与 rightSpeed = base - correction 的差分输出，确保小车在行进过程中能够实时纠偏，维持严格的直线轨迹。

3. 闭环控制流程分析

整个控制逻辑遵循“采样—决策—执行”的闭环循环：系统首先通过 Socket 通讯获取仿真平台下发的位姿反馈，随后在算法中设定目标坐标或角度并计算即时偏差。经过比例环节的数学计算后，生成的 PWM 动力指令经由 setTwoTyres 接口下发至执行机构。这一高频循环确保了小车即使在初始位置随机或存在物理障碍干扰的情况下，依然能以极高的路径跟随精度完成预设任务。

③ 小车路线规划

为了使仿真小车能在 60s 的时限内以最优路径依次穿越四个目标点，本实验构建了一套基于 A*（A-Star）算法的全局路径规划系统。该算法结合了 Dijkstra 算法的完备性与启发式搜索的高效性，能够动态应对随机生成的障碍物环境。

1. 环境建模与地图膨胀

系统首先将仿真环境离散化为 380*220 的二维栅格地图 grid_map。为了确保小车在行进过程中不与障碍物边缘发生物理擦碰，代码引入了障碍物膨胀算法：设定膨胀半径 INFLATION_R 为 14 个单位，利用双重循环将所有障碍物（Barrier）的物理边界向外扩张。这一预处理过程有效地将小车的物理尺寸简化为一个质点，从而将复杂的避障问题转化为在拓扑地图中的安全路径搜寻问题。

2. A*算法启发式估算模型

A* 算法的核心在于代价评价函数 f(n)，它决定了搜索树的扩展方向：

$$f(n) = g(n) + h(n)$$

其中，g(n)为从起点到当前节点 n 的实际代价；h(n)为从当前节点到目标点的预估代价（启发值）。在本代码实现中，启发函数采用曼哈顿距离进行计算：

$$d(A,B) = |x_1 - x_2| + |y_1 - y_2|$$

这种计算方式在栅格地图中具有极高的计算效率，能引导算法优先向目标点方向进行探索。

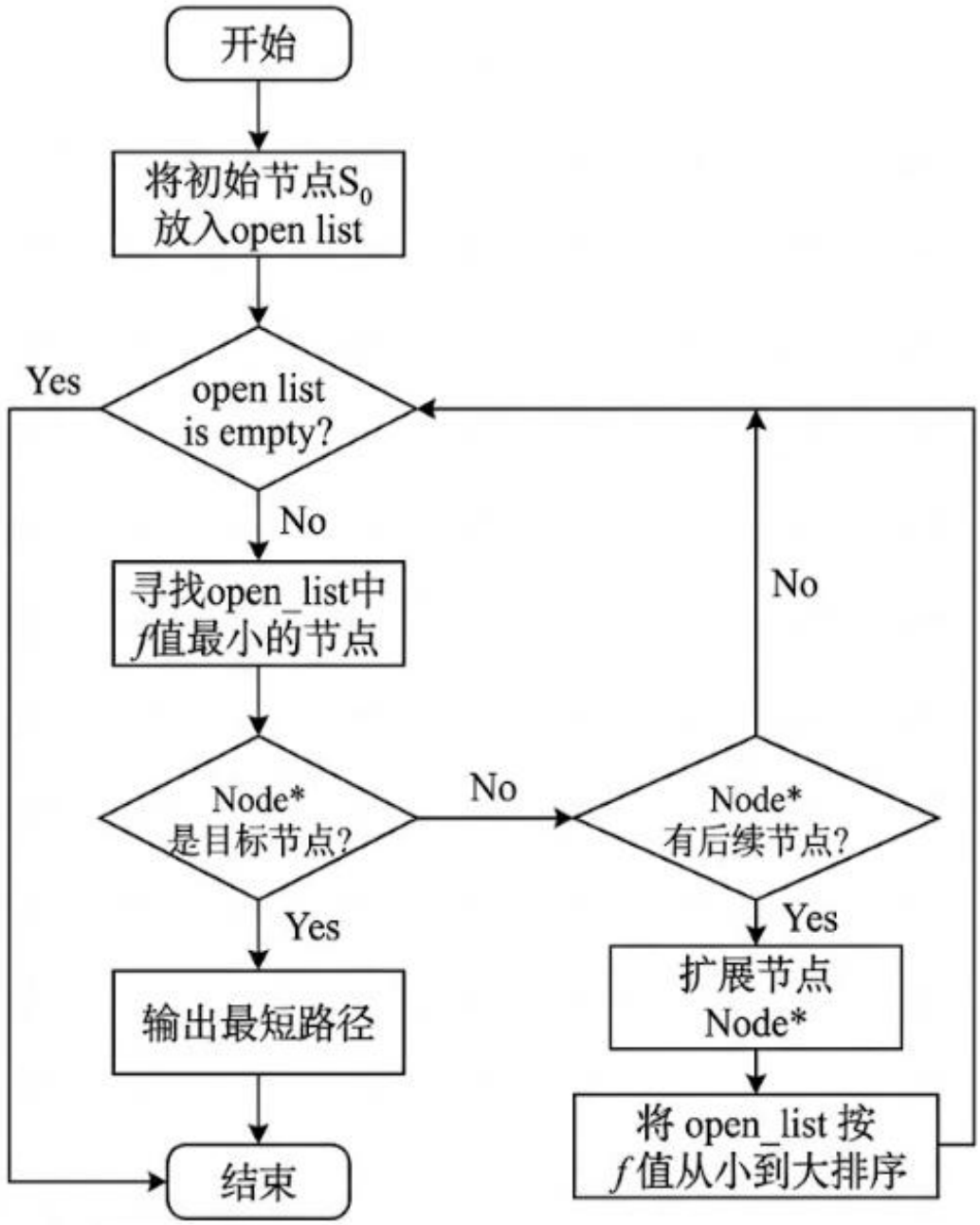

图 30 A*算法流程图

3. 转向惩罚与路径平滑优化

为了避免生成过于细碎、频繁转向的路径（这会导致小车在执行过程中因频繁制动而超时），代码在搜寻逻辑中加入了转向惩罚机制。在计算节点代价时，若当前前进方向与上一节点方向不一致，则在 g(n)中额外累加 20 个单位的代价惩罚，从而诱导算法生成直线占比更高、更易于物理执行的平滑路径。

4. 关键路点提取

由 A*算法生成的原始路径包含大量冗余的逐格坐标。程序通过 getWaypoints 函数对路径执行了矢量化压缩：系统遍历整条路径，仅保留斜率发生变化的转角点作为“关键路点”。最终，状态机调度器依据这些离散的路点驱动 PID 转向控制模块，使小车能够以“段落式直行”的方式高速穿越复杂的障碍区，确保了任务执行的实时性与鲁棒性。

2、 代码分析

编程小车仿真实验代码实现了一个基于状态机调度与 A*路径规划的自动化运动控制系统，主要涵盖了环境建模、路径寻优及高可靠性的闭环运动控制三大核心功能。

核心功能是小车的运动控制逻辑，分别通过 turnToHeading（转向控制）与 moveForward（直线行驶）两个基本动作原语实现。转向部分基于比例控制（P-Control）算法，通过实时计算目标航向与当前 IMU 反馈的误差，精确调制两侧轮胎的转速比以修正角度偏差。移动部分则集成了环境感知安全逻辑：系统在直行过程中会高频轮询 grid_map 数据，一旦检测到碰撞风险或电机堵转（Stall），将立即触发 performRecovery 救车函数，通过主动倒车避险重置运行状态，确保小车在动态障碍物环境下的生存能力。

在初始化阶段，代码通过 buildMap 函数对仿真环境中的障碍物（Barrier）进行解析，并引入了 INFLATION_R（膨胀半径）算法，通过将障碍物边界向外扩张 14 个单位，为小车预留了物理安全冗余。路径规划部分采用了 A*算法，该算法在启发式搜索的基础上加入了“转向惩罚”权重，引导系统生成拐点最少、平滑度最高的最优路径。

系统的整体执行流由一个三段式状态机（0: 规划, 1: 执行, 2: 救车）进行统一调度。代码通过 getWaypoints 方法对原始路径进行矢量化处理，将其拆解为一系列离散的导航路点（Waypoints）。这种设计使程序具备了极强的环境适应能力：当小车到达路点阈值范围后，状态机将自动推进至下一个目标点（Checkpoint），从而在 60 秒的时间限制内，依次精准穿越所有预设位点。

3、 源代码

编程小车仿真实验

```
#include "Controller.h"
#include <algorithm>
#include <chrono>
#include <cmath>
#include <fstream>
#include <iostream>
#include <queue>
#include <thread>
#include <vector>

using namespace std;

Controller* controller = Controller::getInstance();

// ==========================================
// 1. 全局配置 & 数据结构
// ==========================================

const int MAP_W = 380;
const int MAP_H = 220;
const int INFLATION_R = 14;

int grid_map[MAP_H][MAP_W];
```

```
struct Point {
    int x, y;
    bool operator==(const Point& other) const {
        return x == other.x && y == other.y;
    }
    bool operator!=(const Point& other) const { return !(*this == other); }
};

// 状态变量
vector<Point> targets;
bool map_ready = false;
int current_tgt = 0;
int state = 0; // 0: 规划, 1: 执行, 2: 救车
Point next_wp;
bool is_final_leg = false;
Point last_stopped_pos = {-9999, -9999};

// 辅助数学函数
float normalizeAngle(float angle) {
    while (angle > 180)
        angle -= 360;
    while (angle < -180)
        angle += 360;
    return angle;
}

float getShortestAngleDiff(float target, float current) {
    return normalizeAngle(target - current);
}

// ==========================================
// 2. 核心动作控制 (带状态反馈)
// ==========================================

// 简单的倒车动作，用于脱困
void simpleBackwards(float duration_ms) {
    controller->setTwoTyres(-15.0, -15.0);
    int steps = duration_ms / 10;
    for (int i = 0; i < steps; i++)
        Controller::delay(10);
    controller->setTwoTyres(0, 0);
}

// 转弯函数：返回 true 表示成功，false 表示超时失败
bool turnToHeading(float target_angle) {
    target_angle = normalizeAngle(target_angle);
```

```
const float TOLERANCE = 4.0;
const float SPEED = 3.5;
// 增加转弯超时时间限制 (3 秒)
const double TIMEOUT_SEC = 3.0;

float current_rot = controller->getCarMessage().transform.rotation;
if (fabs(getShortestAngleDiff(target_angle, current_rot)) <= TOLERANCE) {
    return true;
}

auto startTime = chrono::steady_clock::now();

while (true) {
    current_rot = controller->getCarMessage().transform.rotation;
    float error = getShortestAngleDiff(target_angle, current_rot);

    if (fabs(error) <= TOLERANCE) {
        // 刹车缓冲
        for (int i = 0; i < 2; i++) {
            controller->setTwoTyres(0, 0);
            Controller::delay(10);
        }
        return true;
    }

    // --- 超时检查 ---
    auto now = chrono::steady_clock::now();
    double elapsed =
        chrono::duration_cast<chrono::milliseconds>(now - startTime)
            .count() /
        1000.0;
    if (elapsed > TIMEOUT_SEC) {
        cout << ">>> [警告] 转弯超时 (>3s)! 放弃转弯." << endl;
        controller->setTwoTyres(0, 0);
        return false; // 返回失败
    }

    // 简单的 P 控制
    float output = (error > 0) ? SPEED : -SPEED;

    // 接近时减速，防止震荡
    if (fabs(error) < 15.0)
        output *= 0.6;
    if (fabs(output) < 1.5)
        output = (error > 0) ? 1.5 : -1.5;
```

```
        if (error > 0)
            controller->setTwoTyres(fabs(output), 0);
        else
            controller->setTwoTyres(0, fabs(output));

        Controller::delay(10);
    }
}

// 直行函数：返回 true 表示到达，false 表示撞墙或堵转
bool moveForward(float distance, float speed) {
    float initX = controller->getCarMessage().transform.position.x;
    float initY = controller->getCarMessage().transform.position.y;
    float currentRot = controller->getCarMessage().transform.rotation;

    // 锁定当前朝向为目标朝向，走直线
    float targetRotation = round(currentRot / 90.0f) * 90.0f;
    targetRotation = normalizeAngle(targetRotation);

    int direction = (distance >= 0) ? 1 : -1;
    distance = fabs(distance);

    auto startTime = chrono::steady_clock::now();
    auto lastCheckTime = startTime;
    float lastX = initX, lastY = initY;

    // 给予每段路程最大时间，防止无限走
    double max_time = (distance / speed) * 3.0;
    if (max_time < 2.0)
        max_time = 2.0;

    while (true) {
        float currX = controller->getCarMessage().transform.position.x;
        float currY = controller->getCarMessage().transform.position.y;
        float currRot = controller->getCarMessage().transform.rotation;

        // 1. 撞墙检测
        int gx = (int)round(currX);
        int gy = (int)round(currY);
        if (gx >= 0 && gx < MAP_W && gy >= 0 && gy < MAP_H) {
            if (grid_map[gy][gx] == 1) {
                cout << ">>> [危险] 撞墙! 停车!" << endl;
                controller->setTwoTyres(0, 0);
                return false;
            }
        }
```

```
float traveled = sqrt(pow(currX - initX, 2) + pow(currY - initY, 2));
float dist_error = distance - traveled;
float angle_error = normalizeAngle(targetRotation - currRot);

// 到达判断
if (dist_error < 8.0) {
    for (int i = 0; i < 2; i++) {
        controller->setTwoTyres(0, 0);
        Controller::delay(10);
    }
    return true;
}

auto now = chrono::steady_clock::now();

// 2. 堵转检测 (每 500ms 检查一次位移)
if (chrono::duration_cast<chrono::milliseconds>(now - lastCheckTime)
        .count() > 500) {
    float d = sqrt(pow(currX - lastX, 2) + pow(currY - lastY, 2));
    // 如果 500ms 内移动小于 0.5 个单位，且距离目标还远，判为堵转
    if (d < 0.5 && dist_error > 10.0) {
        cout << ">>> [警告] 堵转! 无法移动." << endl;
        controller->setTwoTyres(0, 0);
        return false;
    }
    lastCheckTime = now;
    lastX = currX;
    lastY = currY;
}

// 3. 全局超时保护
if (chrono::duration_cast<chrono::seconds>(now - startTime).count() >
    max_time) {
    controller->setTwoTyres(0, 0);
    return false;
}

// 姿态修正 (P 控制走直线)
float base_speed = speed;
float kp_angle = 1.5;
float turn_correction = 0;

if (fabs(angle_error) > 1.0) {
    turn_correction = kp_angle * angle_error;
    if (turn_correction > 8.0)
```

```
                turn_correction = 8.0;
            if (turn_correction < -8.0)
                turn_correction = -8.0;
        }

        float leftSpeed = direction * base_speed + turn_correction;
        float rightSpeed = direction * base_speed - turn_correction;
        controller->setTwoTyres(leftSpeed, rightSpeed);

        Controller::delay(10);
    }
}

// ==========================================
// 3. A* 算法 (保持不变)
// ==========================================

void buildMap(const vector<Barrier>& dynamic_obstacles) {
    for (int y = 0; y < MAP_H; y++)
        for (int x = 0; x < MAP_W; x++)
            grid_map[y][x] = 0;
    for (const auto& obs : dynamic_obstacles) {
        if (obs.type == "Rect") {
            int x1 = max(0, (int)floor(min({obs.point1.x, obs.point2.x,
                                            obs.point3.x, obs.point4.x})));
            int x2 =
                min(MAP_W - 1, (int)ceil(max({obs.point1.x, obs.point2.x,
                                              obs.point3.x, obs.point4.x})));
            int y1 = max(0, (int)floor(min({obs.point1.y, obs.point2.y,
                                            obs.point3.y, obs.point4.y})));
            int y2 =
                min(MAP_H - 1, (int)ceil(max({obs.point1.y, obs.point2.y,
                                              obs.point3.y, obs.point4.y})));
            for (int y = y1; y <= y2; y++)
                for (int x = x1; x <= x2; x++)
                    grid_map[y][x] = 1;
        }
    }
    // 膨胀
    int temp[MAP_H][MAP_W];
    for (int y = 0; y < MAP_H; y++)
        for (int x = 0; x < MAP_W; x++)
            temp[y][x] = grid_map[y][x];
    for (int y = 0; y < MAP_H; y++) {
        for (int x = 0; x < MAP_W; x++) {
            if (grid_map[y][x] == 1) {
```

```
                for (int dy = -INFLATION_R; dy <= INFLATION_R; dy++) {
                    for (int dx = -INFLATION_R; dx <= INFLATION_R; dx++) {
                        int ny = y + dy, nx = x + dx;
                        if (ny >= 0 && ny < MAP_H && nx >= 0 && nx < MAP_W &&
                            grid_map[ny][nx] == 0)
                            temp[ny][nx] = 2;
                    }
                }
            }
        }
    }
    for (int y = 0; y < MAP_H; y++)
        for (int x = 0; x < MAP_W; x++)
            grid_map[y][x] = temp[y][x];
}

struct Node {
    int x, y, g, h, dir;
    bool operator>(const Node& o) const { return g + h > o.g + o.h; }
};
int dist(Point a, Point b) { return abs(a.x - b.x) + abs(a.y - b.y); }

vector<Point> aStar(Point start, Point end) {
    auto findValid = [](Point p) {
        if (p.x >= 0 && p.x < MAP_W && p.y >= 0 && p.y < MAP_H &&
            grid_map[p.y][p.x] == 0)
            return p;
        queue<Point> q;
        q.push(p);
        vector<vector<bool>> vis(MAP_H, vector<bool>(MAP_W, false));
        vis[p.y][p.x] = true;
        int d[4][2] = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};
        while (!q.empty()) {
            Point c = q.front();
            q.pop();
            if (grid_map[c.y][c.x] == 0)
                return c;
            for (auto k : d) {
                int nx = c.x + k[0], ny = c.y + k[1];
                if (nx >= 0 && nx < MAP_W && ny >= 0 && ny < MAP_H &&
                    !vis[ny][nx]) {
                    vis[ny][nx] = true;
                    q.push({nx, ny});
                }
            }
        }
```

```
        return p;
    };

    Point s = findValid(start);
    Point e = findValid(end);

    priority_queue<Node, vector<Node>, greater<Node>> open;
    open.push({s.x, s.y, 0, dist(s, e), 0});

    vector<vector<int>> gScore(MAP_H, vector<int>(MAP_W, 999999));
    vector<vector<Point>> parent(MAP_H, vector<Point>(MAP_W, {-1, -1}));
    gScore[s.y][s.x] = 0;

    int dirs[4][2] = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};
    int dirids[4] = {1, 2, 3, 4};

    while (!open.empty()) {
        Node c = open.top();
        open.pop();
        if (c.x == e.x && c.y == e.y) {
            vector<Point> path;
            Point p = e;
            while (p != s) {
                path.push_back(p);
                p = parent[p.y][p.x];
            }
            path.push_back(s);
            reverse(path.begin(), path.end());
            return path;
        }
        if (c.g > gScore[c.y][c.x])
            continue;

        for (int i = 0; i < 4; i++) {
            int nx = c.x + dirs[i][0], ny = c.y + dirs[i][1];
            if (nx >= 0 && nx < MAP_W && ny >= 0 && ny < MAP_H &&
                grid_map[ny][nx] == 0) {
                // 惩罚转向
                int cost = 1 + (c.dir != 0 && c.dir != dirids[i] ? 20 : 0);
                if (c.g + cost < gScore[ny][nx]) {
                    gScore[ny][nx] = c.g + cost;
                    parent[ny][nx] = {c.x, c.y};
                    open.push(
                        {nx, ny, c.g + cost, dist({nx, ny}, e), dirids[i]});
                }
            }
```

```
        }
    }
    return {};
}

vector<Point> getWaypoints(const vector<Point>& path) {
    if (path.empty())
        return {};
    vector<Point> wps;
    wps.push_back(path[0]);
    if (path.size() < 3) {
        wps.push_back(path.back());
        return wps;
    }
    Point lastD = {path[1].x - path[0].x, path[1].y - path[0].y};
    for (size_t i = 2; i < path.size(); i++) {
        Point curD = {path[i].x - path[i - 1].x, path[i].y - path[i - 1].y};
        if (curD.x != lastD.x || curD.y != lastD.y) {
            wps.push_back(path[i - 1]);
            lastD = curD;
        }
    }
    wps.push_back(path.back());
    return wps;
}

// ==========================================
// 4. 主循环与策略
// ==========================================

// 统一的救车动作
void performRecovery() {
    cout << ">>> [救车模式] 倒车避险..." << endl;

    // 1. 倒车 (盲倒 300ms)
    simpleBackwards(300);

    // 2. 稍微停顿
    Controller::delay(100);

    // 3. 更新当前位置作为最后停止点，防止下次规划立刻选中同一个点
    float cx = controller->getCarMessage().transform.position.x;
    float cy = controller->getCarMessage().transform.position.y;
    last_stopped_pos = {(int)cx, (int)cy};

    // 4. 强制重置状态回 0 (重新寻路)
```

```
    state = 0;
}

void setup() {
    controller->subscribeList = "IMU,RobotBaseTransform";
    controller->getList = "Barrier,CheckPoint";
}

void loop() {
    // 1. 初始化建图
    if (!map_ready) {
        auto bars = controller->getBarrierMessage();
        auto cps = controller->getCheckPointMessage();
        if (bars.empty())
            return;
        cout << ">>> 建图..." << endl;
        buildMap(bars);
        sort(cps.begin(), cps.end(),
             [](auto& a, auto& b) { return a.number < b.number; });
        targets.clear();
        for (auto& c : cps)
            targets.push_back({(int)c.position.x, (int)c.position.y});

        // ==========================================
        //  优化后的初始环境检查 (修复撞墙 Bug)
        // ==========================================
        // ... (在 buildMap(bars); 之后) ...

        // ==========================================
        //  智能环境探测 (前后双向检查)
        // ==========================================
        float sx = controller->getCarMessage().transform.position.x;
        float sy = controller->getCarMessage().transform.position.y;
        float s_rot = controller->getCarMessage().transform.rotation;

        // 定义一个 Lambda 函数来探测特定角度和距离是否有障碍
        // 返回 true 表示有障碍(墙或膨胀区), false 表示安全
        auto checkDirection = [&](float angle_offset, float dist) -> bool {
            float rad = (s_rot + angle_offset) * 3.1415926f / 180.0f;
            int tx = (int)(sx + cos(rad) * dist);
            int ty = (int)(sy + sin(rad) * dist);

            // 1. 检查地图边界
            if (tx < 0 || tx >= MAP_W || ty < 0 || ty >= MAP_H)
                return true;
            // 2. 检查障碍物 (1=墙, 2=膨胀区)
```

```
        if (grid_map[ty][tx] != 0)
            return true;

        return false;
    };

    // 1. 探测前方 (前、左前、右前，只要有一个堵了就算前堵)
    bool front_blocked = checkDirection(0, 18.0) ||
                         checkDirection(20, 18.0) ||
                         checkDirection(-20, 18.0);

    // 2. 探测后方 (正后方)
    bool back_blocked = checkDirection(180, 18.0);

    // 3. 决策逻辑
    if (front_blocked) {
        if (!back_blocked) {
            // [情况 A] 前方堵，后方空 -> 可以安全倒车
            cout << ">>> [策略] 前方受阻，后方安全。执行主动倒车腾出空间..."
                 << endl;

            controller->setTwoTyres(0, 0);
            Controller::delay(100);
            controller->setTwoTyres(-20, -20);
            Controller::delay(600); // 倒车 0.6s
            controller->setTwoTyres(0, 0);
            Controller::delay(200);

            cout << ">>> 倒车完成。" << endl;
        } else {
            // [情况 B] 前后都堵 (夹心状态)
            cout << ">>> [警告] 陷入'夹心'状态！正在测量前后距离..."
                 << endl;

            // --- 1. 测量前方实墙距离 ---
            float dist_front = 999.0;                   // 默认很远
            for (float r = 1.0; r <= 30.0; r += 1.0) { // 向前搜寻 30 个单位
                float rad = s_rot * 3.1415926f / 180.0f;
                int tx = (int)(sx + cos(rad) * r);
                int ty = (int)(sy + sin(rad) * r);

                // 越界或撞到实墙(1)停止
                if (tx < 0 || tx >= MAP_W || ty < 0 || ty >= MAP_H ||
                    grid_map[ty][tx] == 1) {
                    dist_front = r;
                    break;
```

```
                }
            }

            // --- 2. 测量后方实墙距离 ---
            float dist_back = 999.0;
            for (float r = 1.0; r <= 30.0; r += 1.0) { // 向后搜寻 30 个单位
                float rad = (s_rot + 180.0f) * 3.1415926f / 180.0f; // 反向
                int tx = (int)(sx + cos(rad) * r);
                int ty = (int)(sy + sin(rad) * r);

                if (tx < 0 || tx >= MAP_W || ty < 0 || ty >= MAP_H ||
                    grid_map[ty][tx] == 1) {
                    dist_back = r;
                    break;
                }
            }

            cout << ">>> [测距结果] 前方距离: " << dist_front
                 << " | 后方距离: " << dist_back << endl;

            // --- 3. 比较并决策 ---
            // 先停稳
            controller->setTwoTyres(0, 0);
            Controller::delay(100);

            if (dist_front < dist_back) {
                // 离前墙更近 -> 向后退 (去空间大的地方)
                cout << ">>> [策略] 离前墙更近 -> 微量倒车(300ms)" << endl;
                controller->setTwoTyres(-15, -15);
                Controller::delay(300);
            } else {
                // 离后墙更近 (或距离相等) -> 向前挪
                cout << ">>> [策略] 离后墙更近 -> 微量前进(300ms)" << endl;
                controller->setTwoTyres(10, 10); // 正向速度
                Controller::delay(300);
            }

            // 再次停稳，准备进入寻路
            controller->setTwoTyres(0, 0);
            Controller::delay(200);
        }
    } else {
        // [情况 C] 前方开阔 -> 不需要倒车
        if (back_blocked) {
            // [情况 C] 前方虽空，但屁股贴着墙 -> 向前挪一点防止扫尾
            cout << ">>> [策略] "
```

```
                    "前方开阔，但后方受阻。微量向前调整以防车尾撞墙..."
                << endl;

            controller->setTwoTyres(0, 0);
            Controller::delay(100);
            controller->setTwoTyres(15, 15); // 向前
            Controller::delay(400);          // 稍微走一点 (0.4s)
            controller->setTwoTyres(0, 0);
            Controller::delay(200);

            cout << ">>> 向前调整完成。" << endl;
        } else {
            // [情况 D] 前后都开阔 -> 完美状态
            cout << ">>> [策略] 前后视野良好，无需调整，直接出发。" << endl;
        }
    }
    // ==========================================

    map_ready = true;
    return;
}

// ... (以下代码保持不变) ...
if (current_tgt >= targets.size()) {
    cout << "全部完成!" << endl;
    controller->setTwoTyres(0, 0);
    while (1)
        Controller::delay(1000);
}

float cx = controller->getCarMessage().transform.position.x;
float cy = controller->getCarMessage().transform.position.y;
Point target = targets[current_tgt];

if (state == 0) {
    controller->setTwoTyres(0, 0);
    auto raw = aStar({(int)cx, (int)cy}, target);
    if (raw.empty()) {
        cout << "寻路失败! 尝试后退重试..." << endl;
        simpleBackwards(200);
        return;
    }

    auto wps_list = getWaypoints(raw);
    int selected_idx = -1;
    if (wps_list.size() >= 2) {
```

```
            selected_idx = 1;
            for (int i = 1; i < wps_list.size(); i++) {
                float dist_to_last =
                    sqrt(pow(wps_list[i].x - last_stopped_pos.x, 2) +
                         pow(wps_list[i].y - last_stopped_pos.y, 2));
                float dist_to_me = sqrt(pow(wps_list[i].x - cx, 2) +
                                        pow(wps_list[i].y - cy, 2));
                if (i == wps_list.size() - 1) {
                    selected_idx = i;
                    break;
                }
                if (dist_to_me > 5.0 && dist_to_last > 10.0) {
                    selected_idx = i;
                    break;
                }
            }
        }

        if (selected_idx != -1) {
            next_wp = wps_list[selected_idx];
            is_final_leg = (selected_idx == wps_list.size() - 1);
            state = 1;
        } else {
            current_tgt++;
        }
    } else if (state == 1) {
        float dist = sqrt(pow(next_wp.x - cx, 2) + pow(next_wp.y - cy, 2));
        float threshold = is_final_leg ? 10.0 : 15.0;

        if (dist < threshold) {
            controller->setTwoTyres(0, 0);
            last_stopped_pos = next_wp;
            if (is_final_leg) {
                cout << ">>> 到达目标点 " << current_tgt << endl;
                current_tgt++;
                Controller::delay(50);
            }
            state = 0;
            return;
        }

        float angle =
            atan2(-(next_wp.y - cy), next_wp.x - cx) * 180.0 / 3.1415926;
        float snapped_angle = round(angle / 90.0f) * 90.0f;

        bool turn_success = turnToHeading(snapped_angle);
```

```
        if (!turn_success) {
            performRecovery();
            return;
        }

        bool move_success = moveForward(dist, 30.0);
        if (!move_success) {
            performRecovery();
            return;
        }
    }
}
```

4、 小结

本次“编程小车仿真实验”的顺利完成，标志着我从底层的硬件控制逻辑跨越到了复杂的自主路径规划算法开发阶段。通过在 VirtualRobot 仿真系统中的反复迭代，我深刻体会到了UE5物理引擎为算法验证带来的高效性与准确性，使我能够在不受物理场地限制的情况下，快速完成数以百次的路径规划与避障策略调试。在实验过程中，我成功将 A*寻径算法与比例控制（P-Control）逻辑进行了深度整合：利用地图膨胀技术解决了虚拟小车在转弯时的物理碰撞隐患，并通过在代价函数中引入转向惩罚，诱导系统生成了更符合机械执行特性的平滑轨迹。面对起始坐标随机化带来的不确定性，我通过构建三段式状态机，确保了小车即使在极端初始位姿下，也能在 60s 的严苛时限内精准识别并依次穿越四个目标点位。这次仿真实验不仅提升了我对 PID 闭环控制与启发式搜索算法的工程化理解，更让我领悟到“仿真驱动开发”的先进理念——即在虚拟环境下穷尽所有边界逻辑，从而为未来真机系统的鲁棒性提升提供了可靠的理论与算法支撑。

图 31 仿真实验截图

七、挑战任务

1、 任务描述与分析

(1) 题目描述

本项任务核心在于利用“点灯科技”App 作为交互终端，通过蓝牙（Bluetooth）无线通讯协议建立移动设备与 ESP32 主控板之间的双向控制链路，在 1min 的规定时限内，操作者需手动驱动智能小车在布满障碍物的实验空间中执行复杂机动，不仅要精准识别并依次穿越四个预设的目标圆圈，还需在保障行进轨迹流畅性的前提下，将任务耗时缩减至极限。

具体要求：

① 完成智能小车各机械零部件的精密组装，并确保动力电池组与主控系统之间的电气连接稳固可靠；

② 将远程遥控底层控制算法烧录至 ESP32 开发板，并完成开发板与驱动扩展板的集成安装与上电自检；

③ 启动移动端蓝牙模块并实现与小车主控的逻辑配对，在“点灯科技”App 中完成虚拟控制面板的按键映射与 UI 配置；

④ 执行手动遥控任务，在高速移动过程中通过动态轨迹修正实时规避物理障碍，且必须严格遵循预设的逻辑顺序依次通过所有目标点，力求在最短时间内达成全闭环挑战。

(2) 点灯科技远程控制小车

在实验五我们已经得到了完整的小车，将程序烧写进 ESP32 开发板后，使用点灯科技利用蓝牙来与小车连接。

在完成设备配对与 UI 逻辑映射后，系统构建了涵盖按钮、滑块与摇杆在内的三位一体人机交互接口。按钮区域作为基础运动指令的下发中枢，集成了“左轮转”、“直走”及“右轮转”三个原子动作组件，主要负责执行长距离巡航与快速姿态换向等离散运动逻辑。为了克服物理电机在实际运行中因机械摩擦或供电波动产生的偏航现象，系统引入了滑块区域执行精细化的动力平衡修正；操作者通过动态调节滑块数值，可实现对左右轮输出转速的实时差速补偿，从而在物理层面保障小车在高速直线行驶中的轨迹鲁棒性。

在应对复杂障碍物环境下的精细机动需求时，摇杆区域提供了更高维度的连续向量控制能力。相较于按钮模式的固定步进，摇杆可根据操作者的触控偏移量，实时解算出平滑的速度梯度与转向曲率，使小车能够执行弧线过弯等非线性机动，大幅提升了在密闭障碍区内的机动效率。在实际执行挑战任务的逻辑流中，操作者通常采用“复合操控策略”：利用“直走”按钮配合滑块修正维持基础航向，当检测到目标圆圈或障碍物边缘时，通过侧向按钮执行紧急避障动作，最后利用摇杆进行末端轨迹的微量微调。这种“按键调向、滑块调平、摇杆精控”的多维交互模型，确保了系统能够在 1min 的时限阈值内，以最优化的动力响应完成所有预设目标点位的物理穿越。

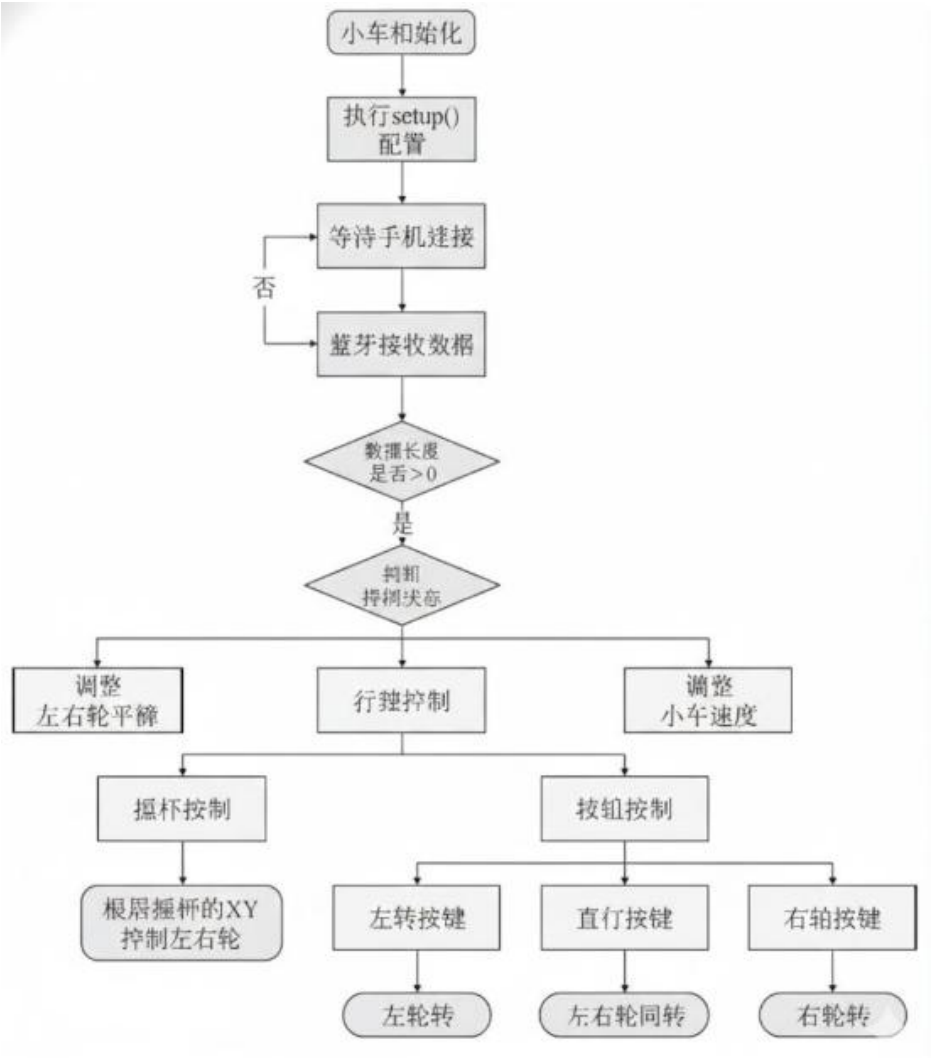

图 32 遥控小车流程图

(3) 小车运动原理

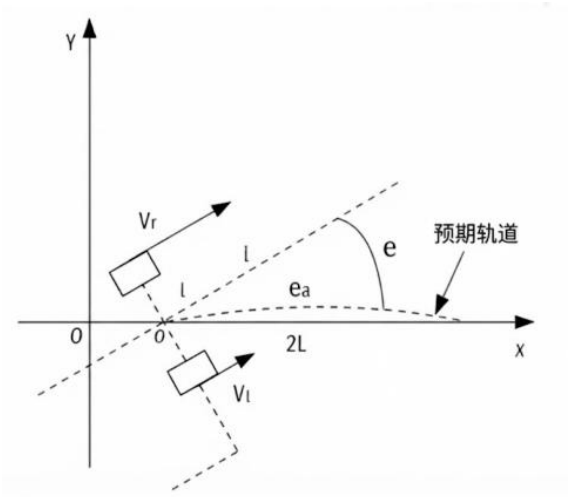

图 33 小车差速驱动分析

本实验智能小车的动力学核心基于差速驱动原理（Differential Drive），即通过独立调节左右两侧驱动轮的瞬时线速度 v1（左轮速度）与 v2（右轮速度），实现对运动轨迹与航向角的精密调控。在实际执行逻辑中，系统根据两侧车轮的速度矢量关系，主要分为以下三种运动模态：首先，当两侧车轮速度大小相等且方向一致（v1 = v2）时，小车将维持稳定的直线行进轨迹；其次，当两侧车轮速度大小相等但方向相反（v1 = -v2）时，小车将绕其几何中心执行原位旋转，这一特性赋予了系统在狭窄空间内极佳的机动性能；最后，当两侧速度不一致（v1！=v2）时，小车将向速度较低的一侧发生偏转，从而实现转向动作。

从运动学几何约束角度分析，小车偏转时的转弯半径 R 与两侧速度差及轮距 H 满足特定的非线性约束关系。当小车向右偏转（v1 ＞ v2）或向左偏转（v1 ＜ v2）时，其中心轨迹的转弯半径 R 可由下式推导得出：

$$R = \frac{H}{2} \times \frac{v_1 + v_2}{v_1 - v_2}, v_1 \neq v_2$$

这种基于转速差异实现转向的技术方案，允许系统在行进过程中动态修正航向角，既能完成大半径的平滑过弯，也能在极小半径内执行紧急避障。在程序实现层面，通过 ESP32 对电机驱动模块输出不同占空比的 PWM 信号，即可实时改变 v1 与 v2 的参数组合，从而构建起从小车底层动力响应到上层复杂路径规划的物理反馈闭环。

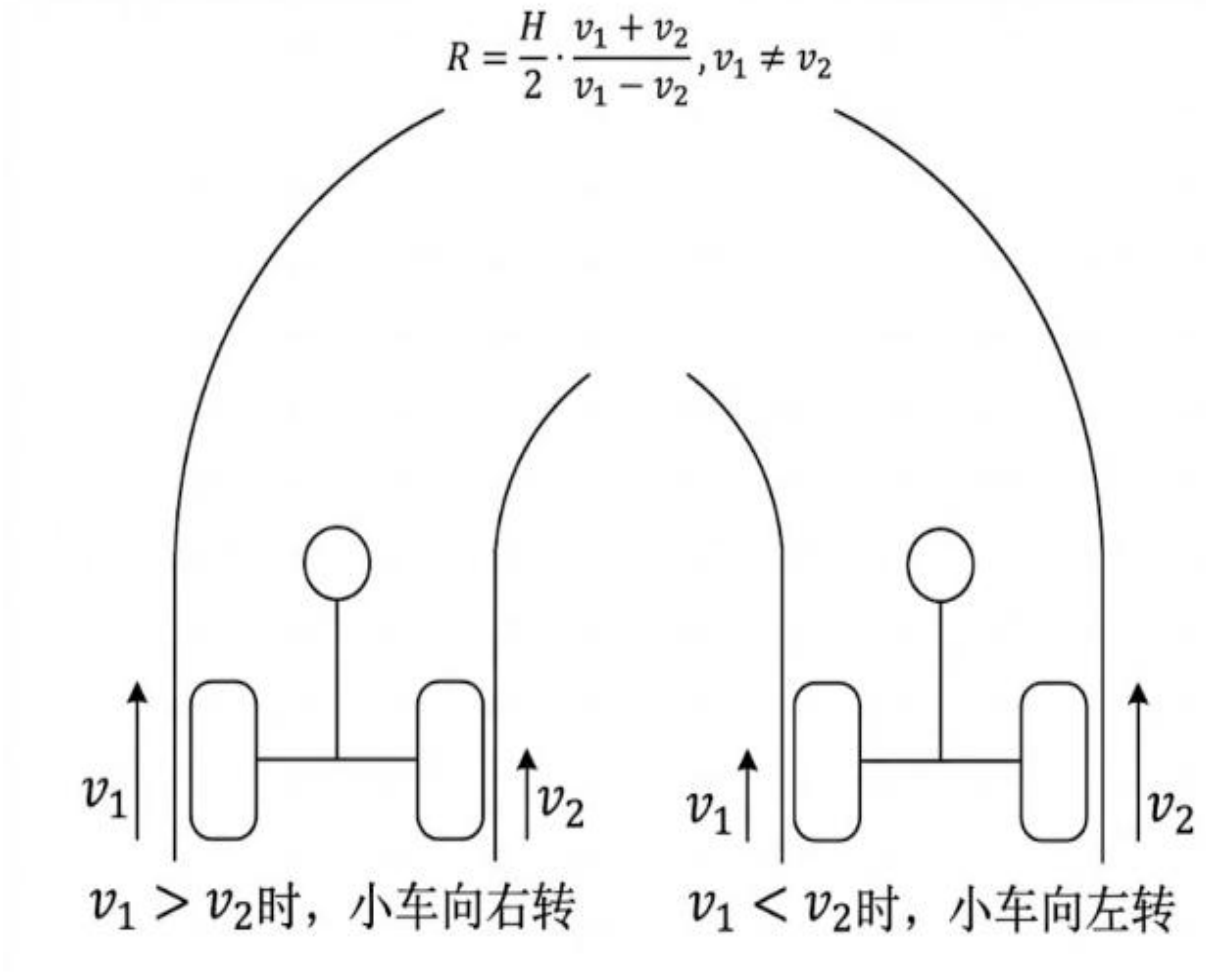

图 34 差速机器人运动轨迹几何模型图

(4) 实验场地介绍

本次“挑战任务”的实地验证环节于华南农业大学数学与信息学院 701 实验室门前空地展开。场地构建采用工业级铝型材围合而成，形成了一个标准的矩形封闭式物理竞赛空间，旨在为智能小车的运动规划提供明确的几何边界约束。在环境复杂度的配置层面，场地内策略性地布设了 6 块由 EPE 珍珠棉材料制成的矩形泡沫块作为物理障碍物；这些障碍物以离散分布的形式构成了非连续的行进通道，极大考验了操作者在手动遥控模式下的瞬时避障决策与路径规划效率。此外，在既定的实验平面内，分别放置了蓝、黄、红、青四种高对比度的目标圆圈点位。要求开发者依据预设逻辑顺序，引导小车在复杂的障碍密度中完成精准的定点穿越。整体场地布局实现了机械稳固性、静态避障逻辑与多目标识别任务的高度集成，为检验 ESP32 主控系统在实战环境下的指令解析精度与动力响应速度提供了标准化的物理基准环境。

图 35 挑战任务地图

2、 代码分析

本程序基于 Blinker 协议栈构建，利用 ESP32 的 BLE 蓝牙模块与 ledc 硬件外设实现了远程遥控与动力驱动的深度集成。在控制逻辑层面，系统通过 Joystick_callback 异步回调函数实时解析摇杆坐标，并利用差速驱动模型动态解算左右轮的 PWM 占空比，赋予了小车卓越的非线性转向与避障机动能力。为了克服物理电机的机械偏差，程序引入了基于 setPWM()的动力补偿算法，通过滑块（Balance）实时调节双侧电机的转速权重，从算法层面保障了高速巡航时的直线轨迹鲁棒性。此外，按键模式通过 BLINKER_CMD_BUTTON_PRESSED 指令提供了离

散的“长按启动、松开即停”功能，配合 500Hz 的高频 PWM 输出，确保了小车在执行挑战任务穿越目标点时的动作精准度与响应实时性。

3、 源代码

挑战任务——蓝牙遥控小车

```
#define BLINKER_BLE
#define LSPEED 16 // 左轮速度引脚
#define LDIREC 14 // 左轮方向引脚
#define RSPEED 27 // 右轮速度引脚
#define RDIREC 17 // 右轮方向引脚
#define JOY 0
#define PRESS 1
#include <Blinker.h> // 引入 Blinker 库
// 模式按钮
BlinkerButton Button("PATTERN");
// 启动按钮
BlinkerButton ButtonL("StartL");
BlinkerButton ButtonR("StartR");
// PWM 滑块
#define BALANCE_RANGE 100
BlinkerSlider SliderL("Balance"); // 微调左右轮的差异，使左右轮能走直线
BlinkerSlider SliderR("PWM"); // 值越大电机最大转速就越大
// 方向按钮
BlinkerButton ButtonZ("DirecL");
BlinkerButton ButtonY("DirecR");
// 摇杆
BlinkerJoystick Joystick("joy");
// PWM 参数
int channel_L = 0;        // 左轮 PWM 通道
int channel_R = 1;        // 右轮 PWM 通道
int freq = 500;           // PWM 频率
int resolution_bits = 10; // 0~1023
int PWM_L = 223;          // 记录 左轮
int PWM_R = 223;          // 记录 右轮
// 模式[1 长按模式][0 摇杆模式]
int pattern = JOY; // 两种模式，摇杆模式， 按键模式
int balance = 50;  // 居中
int pwmValue = 0;

// 回调函数
void Joystick_callback(uint8_t xAxis, uint8_t yAxis) {
    if (pattern == JOY) {
        if (xAxis == 128 && yAxis == 128) // 归位停止
        {
            ledcWrite(channel_L, 0);
            ledcWrite(channel_R, 0);
```

```
            return;
        } else {
            float distance = sqrt((xAxis - 128) * (xAxis - 128) +
                                  (yAxis - 128) * (yAxis - 128));

            if (yAxis < 127) // 摇杆向上
            {
                digitalWrite(LDIREC, HIGH);
                digitalWrite(RDIREC, HIGH);
                if (xAxis < 128) {
                    ledcWrite(channel_L, (128 - yAxis) * PWM_L / 128);
                    ledcWrite(channel_R, PWM_R * distance / 128);
                } else {
                    ledcWrite(channel_L, PWM_L * distance / 128);
                    ledcWrite(channel_R, (128 - yAxis) * PWM_R / 128);
                }
            } else // 后退
            {
                digitalWrite(LDIREC, LOW);
                digitalWrite(RDIREC, LOW);
                if (xAxis < 128) {
                    ledcWrite(channel_L, (yAxis - 128) * PWM_L / 128);
                    ledcWrite(channel_R, PWM_R * distance / 128);
                } else {
                    ledcWrite(channel_L, PWM_L * distance / 128);
                    ledcWrite(channel_R, (yAxis - 128) * PWM_R / 128);
                }
            }
        }
    }
}

void Button_callback(const String& state) {
    pattern = (pattern + 1) % 2;
    if (pattern == JOY) {
        Button.text("JOY");
        Button.print();
    } else if (pattern == PRESS) {
        Button.text("PRESS");
        Button.print();
    }
}
void ButtonL_callback(const String& state) {
    // 长按
    if (state == BLINKER_CMD_BUTTON_PRESSED) {
        ledcWrite(channel_L, PWM_L);
```

```
    }
    // 松开
    else {
        ledcWrite(channel_L, 0);
    }
}
void ButtonR_callback(const String& state) {
    // 长按
    if (state == BLINKER_CMD_BUTTON_PRESSED) {
        ledcWrite(channel_R, PWM_R);
    }
    // 松开
    else {
        ledcWrite(channel_R, 0);
    }
}

void setPWM() {
    // 设置 PWM_L
    PWM_L = 223 + pwmValue * 8 + BALANCE_RANGE * (balance - 50) / 50;
    if (PWM_L > 1023)
        PWM_L = 1023;
    // 设置 PWM_R
    PWM_R = 223 + pwmValue * 8;
}

void SliderL_callback(int32_t value) {
    balance = value;
    setPWM();
}

void SliderR_callback(int32_t value) {
    pwmValue = value;
    setPWM();
}

void ButtonZ_callback(const String& state) {
    // 设置 LDIREC，改变按钮文本
    digitalWrite(LDIREC, !digitalRead(LDIREC));
    if (digitalRead(LDIREC)) {
        ButtonZ.text("F");
        ButtonZ.print();
    } else {
        ButtonZ.text("B");
        ButtonZ.print();
    }
```

```
}

void ButtonY_callback(const String& state) {
    // 设置 LDIREC，改变按钮文本
    digitalWrite(RDIREC, !digitalRead(RDIREC));
    if (digitalRead(RDIREC)) {
        ButtonY.text("F");
        ButtonY.print();
    } else {
        ButtonY.text("B");
        ButtonY.print();
    }
}

void setup() {
    Serial.begin(115200);
    BLINKER_DEBUG.stream(Serial);
    pinMode(LDIREC, OUTPUT); // 左轮方向引脚
    pinMode(RDIREC, OUTPUT); // 右轮方向引脚

    // 设置向前
    digitalWrite(LDIREC, HIGH);
    digitalWrite(RDIREC, HIGH);

    // 设置 PWM 通道 并连接对应引脚
    ledcSetup(channel_L, freq, resolution_bits); // 设置左轮 PWM 通道
    ledcSetup(channel_R, freq, resolution_bits); // 设置右轮 PWM 通道

    // 将通道与对应的引脚连接
    ledcAttachPin(LSPEED, channel_L);
    ledcAttachPin(RSPEED, channel_R);

    Blinker.begin();
    // 左轮控制按钮
    ButtonL.attach(ButtonL_callback);
    ButtonZ.attach(ButtonZ_callback);
    // 右轮控制按钮
    ButtonR.attach(ButtonR_callback);

    SliderL.attach(SliderL_callback);
    SliderR.attach(SliderR_callback);

    ButtonY.attach(ButtonY_callback);
    // 模式按钮
    Button.attach(Button_callback);
    // 摇杆
```

```
    Joystick.attach(Joystick_callback);
}
void loop() { Blinker.run(); }
```

4、 小结

本次“蓝牙遥控小车挑战任务”作为整篇实验报告的压轴环节，是我在嵌入式开发与移动机器人控制学习过程中印象最为深刻的一场实战演练。实验初期，由于 TT 减速电机的物理参数微差，小车在巡航过程中出现了明显的“行驶偏移”现象，我通过在数学与信息学院 701 实验室场地的反复路测，利用程序中的 setPWM 动力补偿逻辑与 App 端的滑块控件，成功实现了对左右轮转速比的线性微调，从根本上攻克了直线行驶的稳定性难点。在操控技巧的磨合阶段，我经历了从初期频繁撞击障碍物到能够熟练运用摇杆执行“甩尾过弯”的技术跨越，通过对实验场地几何布局的深度建模与肌肉记忆的反复强化，最终创造了 34s 穿越全部 4 个目标点位的最佳记录。这次任务不仅让我深入理解了低功耗蓝牙（BLE）在人机交互系统中的实时响应特性，更让我切身体会到理论建模与工程实践相结合的深远意义。在面对随机障碍物与极限时限的双重压力下，我不仅提升了复杂逻辑下的问题解决能力，更激发了对智能控制技术的浓厚研究兴趣，为未来探索更高级别的自主导航算法奠定了坚实的实战基础。

四、总结

在本学期的《机器人技术》课程学习中，我系统地完成了从底层硬件驱动到高层控制算法的全栈式开发训练。通过这一系列循序渐进的实验项目，我不仅深化了对嵌入式系统的理解，更掌握了将理论模型转化为工程实践的关键方法。以下是各实验阶段的总结：

① LED 模拟交通灯实验

作为对 ESP32 开发板的初次实践探索，本实验不仅让我掌握了 GPIO 接口的模式配置与电平调度，更让我理解了嵌入式编程中逻辑时序的重要性。通过在代码中精确编写高低电平的切换逻辑，我成功实现了红绿黄三色灯的交替闪烁，这一过程让我对硬件底层的“指令执行-物理响应”机制有了直观的认知，为后续复杂外设的驱动奠定了坚实基石。

② 模拟开关门实验

在本环节中，我重点攻克了舵机的精确角度控制技术。通过调制 PWM（脉宽调制）信号的占空比，我能够精准驱动舵机旋转至预设角度以模拟大门的开关动作。实验中最大的收获在于尝试了多线程（Multi-threading）控制逻辑，实现了在不阻塞主循环的前提下同时调度舵机与 LED 状态指示灯，这种并发控制思维对提升机器人的系统响应速度至关重要。

③ 测距与通讯实验

本实验通过 HC-SR04 超声波模块的声波反射原理，计算发射与接收的时间差，并将其换算为高精度的距离参数。此外，我成功实现了两块 ESP32 开发板之间的无线通讯，这一突破性进展让我理解了分布式感知系统中数据交互的流程，学会了如何通过主从机配合实现复杂的环境探测任务。

④ 远程控制小灯实验

该实验展示了物联网（IoT）技术的强大交互能力。通过集成“点灯科技”App 与 ESP32 的网络连接协议，我实现了跨局域网的设备唤醒与状态监控。这种基于云端的远程操控逻辑非常有趣，不仅让我看到了物联网技术的雏形，更直接为后续挑战任务中小车的远程蓝牙操控策略提供了关键的技术雏形与实操经验。

在完成上述基础铺垫后，实验环节正式进入了最具挑战性的小车实战阶段：

⑤ 小车组装与测试实验

这是所有实验中看似最简单实则最具工程意义的一环。通过从零开始的硬件搭建，我全面解构了移动机器人的机械结构，包括底盘刚性、电机传动比以及 L293D 驱动板的电气连接。实验中，我重点研究了差速控制原理，理解了如何通过微调两侧轮速来实现直线修正与圆弧转向，这让我从小车的“组建者”转变为其运动逻辑的“设计者”。

⑥ **编程小车仿真实验**

本实验将算法开发推向了高潮。在基于 UE5 引擎构建的 VirtualRobot 仿真平台上，我利用 C++开发了一套集成 A*寻径算法与 PID 闭环控制的智能驾驶系统。在 24 小时在线的高保真仿真环境中，我反复优化路径规划逻辑，确保虚拟小车能以最优姿态避开障碍物，并依次精准识别并穿越四个目标位点，这段深度的算法调试经历极大地提升了我的逻辑建模能力。

⑦ **挑战任务——手动操控小车**

该任务堪称仿真实验的“现实落地版”，要求在真实物理赛道中手动规避障碍并执行计时穿越任务。在数信院楼 701 实验室门口的赛道上，我通过数十次的调试逐渐形成了“肌肉记忆”，成功克服了电机物理偏航等工程误差。虽然在平时的强化练习中我曾创造出 34s 完成全地图穿越的记录，整个过程让我彻底领悟了“理论与实践结合”的深刻含义。

纵观整个学期的学习，《机器人技术》课程不仅涵盖了力学与运动学的理论模型，更侧重于将算法部署至真实嵌入式环境的工程能力培养。从最初面对复杂接线时的生涩，到最后能熟练运用 A*算法解决实际路径规划问题，我深刻领悟到“仿真验证先行，真机调试跟进”的科学开发流程。2025 年的《机器人技术》课程现已圆满结束，但我在本课程中所建立的软硬件协同开发逻辑与面对复杂工程问题的攻关意识，将持续为我后续的科研与竞赛之路提供助力。

评分表

学号：	姓名：

评分项目　　分数

1．任务目标达成度（50 分）
2．报告是否工整美观（20 分）
3．所学知识在设计中的运用是否得当（20 分）
4．对设计要求较高的部分是否有更深的思考（10 分）

总成绩：

评语：

评阅老师：　　评阅时间：　　年　　月　　日